Namespace LlamaLogic.Architecture

Classes

Blueprint
Represents a tray blueprint (a `.blueprint` file): a lot's objects and architecture (🔓)

BlueprintImage
Represents an image stored in a Blueprint (🔓)

BlueprintObjectDefinitionMapping
Represents a pair of object definition ids stored in a Blueprint (🔓)

FloorFeatureLayer
Represents a layer of the FloorFeaturesSection: one type of floor feature, with a set of values per level

FloorFeatureLevel
Represents the values of a FloorFeatureLayer on one level

FloorFeaturesSection
Represents the FloorFeatures section: the floor feature layers (🔓)

FloorIdGridSection
Represents one of the seven floor id grids: the FloorKeysSection id painted on each quarter of each tile (🔓)

FloorKeyEntry
Represents an entry of the FloorKeysSection: a floor pattern key and the id the floor id grids use for it

FloorKeysSection
Represents the FloorKeys section: the floor pattern keys the floor id grids refer to by id (🔓)

FloorRotationGridSection
Represents one of the five floor rotation grids: the rotation of the pattern on each quarter of each tile of a FloorIdGridSection (🔓)

LegacyFencePoint
Represents an entry of the LegacyFencePointsSection

LegacyFencePointsSection
Represents the LegacyFencePoints section: two lists of points with fence and ceiling rail keys, found only in older world files (🔓)

LegacyWallGraphEdge
Represents an edge of Edges

LegacyWallGraphEdgePart
Represents a part of a LegacyWallGraphEdge

LegacyWallGraphProductEdge
Represents an edge of ProductEdges1 or ProductEdges2

LegacyWallGraphSection
Represents the LegacyWallGraph section: a graph of wall edges with their wall patterns and products, found only in older world files (🔓)

LotArchitecture
Represents the architecture of a lot: its walls, floors, roofs, stairs, terrain and the rest of its build, as typed sections (🔓)

LotArchitectureSection
Represents one section of a lot's architecture: one record of the lot's RecordBundle

LotInfoPlexValue
Represents one entry of the plex map of a LotInfoSection: an optional value per plex unit

LotInfoSection
Represents the LotInfo section: the lot's levels, style, blueprint and users (🔓)

LotInfoUser
Represents one of the two user records of a LotInfoSection

LotObjectsSection
Represents the LotObjects section: the objects of the lot, as an `EA.Sims4.Network.LotObjectList` message (🔓)

LotSizeSection
Represents the LotSize section: the lot's dimensions (🔓)

ModularPiece
Represents a modular piece of the ModularPiecesSection

ModularPiecesSection
Represents the ModularPieces section: the modular pieces placed on the lot (🔓)

ObjectFallbacksSection
Represents the ObjectFallbacks section: the game's object fallback data, four sets of object ids (🔓)

ObjectTileBounds
Represents one entry of an ObjectTileBoundsSection: the tiles an object covers

ObjectTileBoundsSection

Represents the ObjectTileBounds section: the tile rectangles of the lot's objects (🔓)

PendingPlacementSection

Represents the PendingPlacement section: a footprint the game places on the lot after loading (🔓)

Plex

Represents a plex (a unit of a multi-unit lot) of the PlexesSection, with the blocks that make up its outline

PlexGroup

Represents a group of the PlexGroupsSection

PlexGroupMap

Represents an entry of the map from which the game looks up groups of the PlexGroupsSection

PlexGroupMapEntry

Represents an entry of a PlexGroupMap

PlexGroupsSection

Represents the PlexGroups section: groups of values the PlexesSection refers to by id (🔓)

PlexesSection

Represents the Plexes section: the lot's plexes (units), with the blocks of their outlines (🔓)

Pond

Represents a pond of the PondsSection

PondCell

Represents a tile of a Pond and the quarters of it the pond covers

PondsSection

Represents the Ponds section: the lot's ponds and the tile quarters each one covers (🔓)

ProductTable

Represents a table of catalog product keys and the ids by which the rest of a section refers to them

ProductTableEntry

Represents an entry of a ProductTable: a catalog product key and its id

ResourceManifestSection

Represents the ResourceManifest section: the keys of the catalog resources the other sections reference (🔓)

RevisionSection

Represents the Revision section: the lot's revision counter (🔓)

Roof
Represents one roof of a RoofsSection

RoofsSection
Represents the Roofs section: the lot's roofs (🔓)

Stair
Represents one stair or ladder of a StairsSection

StairStage
Represents one stage of a Stair: a straight run of steps

StairsSection
Represents the Stairs section: the lot's stairs and ladders (🔓)

TerrainHeightsSection
Represents the TerrainHeights section: the lot's terrain height grid, a second vertex grid of unknown role and a water level per tile quarter (🔓)

TerrainPaintLayer
Represents a layer of the TerrainPaintSection: a terrain paint and the mask of where it is applied

TerrainPaintSection
Represents the TerrainPaint section: the terrain paint layers, each a paint and a mask of where it is applied (🔓)

Unknown7B33C1F5Section
Represents the Unknown7B33C1F5 section: two lists of 64-bit values whose meaning has not been established (🔓)

UnknownC9106475Entry
Represents one entry of an UnknownC9106475Section

UnknownC9106475Level
Represents one level of an UnknownC9106475Entry

UnknownC9106475Section
Represents the UnknownC9106475 section: a list of entries, each holding levels with keys, whose meaning has not been established (🔓)

VenueBusinessInfo
Represents one business entry of a VenueSection

VenueSection

Represents the Venue section: the lot's venue (🔓)

VertexGraph

Represents one of the graphs of vertices and edges of the VertexGraphsSection

VertexGraphEdge

Represents an edge of a VertexGraph

VertexGraphVertex

Represents a vertex of a VertexGraph

VertexGraphsSection

Represents the VertexGraphs section: graphs of vertices and edges, whose version decides the layout of the WallEdgesSection (🔓)

WallBlock

Represents a block: a room, platform, wall line, fence line, pool or fountain, as loops of segments on one level, with the catalog products along them

WallBlockLoop

Represents a closed or open polygon of a WallBlock, as its segments

WallBlockSegment

Represents a segment of a WallBlockLoop: the run from one point of the loop to the next, with the catalog products along it

WallBlocksSection

Represents the WallBlocks section: the lot's blocks, which are its rooms, wall lines, fence lines, pools and fountains (🔓)

WallEdgePatterns

Represents an entry of WallPatternEdges: the two wall patterns of a wall edge

WallEdgeProduct

Represents an entry of ProductEdges or AdditionalProductEdges: the catalog product along a wall edge

WallEdgesLegacyEntry

Represents an entry of LegacyEntries

WallEdgesSection

Represents the WallEdges section: the wall patterns and catalog products along the lot's wall edges (🔓)

WeatherSection

Represents the Weather section: the lot's seasonal weather interpolation value (🔓)

WorldLotObjectsModel

Represents a WorldLotObjects resource: the objects of a lot template in a world file, as a `LotObjectList` protobuf message in protobuf text format (🔓)

ZoneArchitecture

Provides access to the architecture of the lots of a zone in a save game: the `architecture_data` field of the `EA.Sims4.Persistence.ZoneObjectData` protobuf message, the content of a ZoneObjectData resource

Structs

RoofPoint

Represents a lot-local tile position of a Roof's footprint

Enums

FloorQuarter

The four triangular quarters of a tile, named by the side of the tile each one touches, in the order the game stores them

LotArchitectureSectionType

The types of the sections of a lot's architecture, each of which is one record of the lot's RecordBundle

WallBlockSegmentKind

Specifies the kind of a WallBlockSegment

WallBlockType

Specifies the type of a WallBlock

Class Blueprint

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a tray blueprint (a `.blueprint` file): a lot's objects and architecture (🔓)

```
public sealed class Blueprint
```

Inheritance

object ← Blueprint

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The file is a 32-bit version and the length of its content, followed by the content compressed with Snappy (the raw block format). The content is a header, the LotObjects as a protobuf message, ObjectDefinitionMappings, Images and finally the Architecture as a record bundle. Blueprints without the header (HasHeader is false) hold only the objects and the architecture; Sims 4 Studio reads them, but none has been observed.

Snappy compressors differ in their output, so Encode() reuses the compressed bytes it decoded when the content it would write is unchanged, and otherwise compresses the content anew. Decoding and encoding a blueprint the game wrote therefore reproduces it byte for byte.

Properties

Architecture

Gets/sets the lot's architecture

```
public LotArchitecture Architecture { get; set; }
```

Property Value

LotArchitecture

ContentVersion

Gets/sets the version of the content's header, which is 2 in every blueprint observed and decides whether the content holds Images

```
public int ContentVersion { get; set; }
```

Property Value

int

HasHeader

Gets/sets whether the content begins with a header, which every blueprint observed does

```
public bool HasHeader { get; set; }
```

Property Value

bool

Images

Gets the images, which only content version 2 holds

```
public Collection<BlueprintImage> Images { get; }
```

Property Value

Collection<BlueprintImage>

LotObjects

Gets/sets the lot's objects

```
public LotObjectList LotObjects { get; set; }
```

Property Value

LotObjectList

ObjectDefinitionMappings

Gets the pairs of object definition ids, which only blueprints with a header hold

```
public Collection<BlueprintObjectDefinitionMapping> ObjectDefinitionMappings { get; }
```

Property Value

Collection<BlueprintObjectDefinitionMapping>

Unknown1

Gets/sets a value of the header that follows the objects, which is 1 in every blueprint observed

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Version

Gets/sets the version of the file, which is 1 in every blueprint observed

```
public int Version { get; set; }
```

Property Value

int

Methods

Decode(ReadOnlyMemory<byte>)

Decodes a blueprint from the content of its file

```
public static Blueprint Decode(ReadOnlyMemory<byte> data)
```

Parameters

`data` ReadOnlyMemory<byte>

Returns

Blueprint

Exceptions

InvalidDataException

The data is not a well-formed blueprint, or its architecture cannot be decoded

Encode()

Encodes the blueprint as the content of its file

```
public ReadOnlyMemory<byte> Encode()
```

Returns

ReadOnlyMemory<byte>

Exceptions

InvalidOperationException

Architecture or LotObjects is null, the architecture cannot be encoded as it is, or the content is too large for the format

Class BlueprintImage

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an image stored in a Blueprint (🔓)

```
public sealed class BlueprintImage
```

Inheritance

object ← BlueprintImage

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Every image observed is a JPEG. In every blueprint observed, the low 56 bits of Id are a counter unique within the blueprint and its high byte is 0x10, 0x30 or 0x50; what the id identifies has not been established.

Constructors

BlueprintImage(ulong, ReadOnlyMemory<byte>)

Initializes a new instance of the BlueprintImage class

```
public BlueprintImage(ulong id, ReadOnlyMemory<byte> data)
```

Parameters

`id` ulong

`data` ReadOnlyMemory<byte>

Properties

Data

Gets/sets the content of the image

```
public ReadOnlyMemory<byte> Data { get; set; }
```

Property Value

ReadOnlyMemory<byte>

Id

Gets/sets the id of the image

```
public ulong Id { get; set; }
```

Property Value

ulong

Class BlueprintObjectDefinitionMapping

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a pair of object definition ids stored in a Blueprint (🔓)

```
public sealed class BlueprintObjectDefinitionMapping
```

Inheritance

object ← BlueprintObjectDefinitionMapping

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

In every blueprint observed, From is the object definition id of one of the blueprint's LotObjects, and To usually is not. Hypothesis: To is the object definition the game substitutes when From is unavailable (Sims 4 Studio calls these object fallbacks).

Constructors

BlueprintObjectDefinitionMapping(ulong, ulong)

Initializes a new instance of the BlueprintObjectDefinitionMapping class

```
public BlueprintObjectDefinitionMapping(ulong from, ulong to)
```

Parameters

`from` ulong

`to` ulong

Properties

From

Gets/sets the object definition id of one of the blueprint's lot objects

```
public ulong From { get; set; }
```

Property Value

ulong

To

Gets/sets the object definition id paired with From

```
public ulong To { get; set; }
```

Property Value

ulong

Class FloorFeatureLayer

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a layer of the FloorFeaturesSection: one type of floor feature, with a set of values per level

```
public sealed class FloorFeatureLayer
```

Inheritance

object ← FloorFeatureLayer

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader.

Properties

Levels

Gets the levels, in the order they are stored, which is ascending in the lots examined

```
public Collection<FloorFeatureLevel> Levels { get; }
```

Property Value

Collection<FloorFeatureLevel>

Remarks

A level of a layer of type 0 or 1 has one value per tile (SizeX and SizeZ are the lot's); a level of a layer of type 2 has a single value (both are 1).

Type

Gets/sets the type of the layer, 0 through 2

```
public byte Type { get; set; }
```

Property Value

byte

Remarks

The game rejects other types as an "Invalid layer type"; the names of the three types are not in the game's code. In the lots examined the values of type 0 are 0 or 255, those of type 1 are 0, 127, 128 or 255, and those of type 2 are 0; layers of type 2 have no levels in all but 6 records.

Exceptions

ArgumentOutOfRangeException

The type is not 0 through 2

Version

Gets/sets the version of the layer

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

The game's reader accepts 0 and 1 and writes 1, the only version in the lots examined; both have the same layout.

Exceptions

ArgumentOutOfRangeException

The version is not 0 or 1

Class FloorFeatureLevel

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the values of a FloorFeatureLayer on one level

```
public sealed class FloorFeatureLevel
```

Inheritance

object ← FloorFeatureLevel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The values are stored as SizeZ rows of SizeX bytes, unlike the floor grids. The game's `get_floor_feature` returns a value divided by 255.

Constructors

FloorFeatureLevel(int, int)

Initializes a new instance of the FloorFeatureLevel class with every value 0

```
public FloorFeatureLevel(int sizeX, int sizeZ)
```

Parameters

`sizeX` int

The number of tiles along the lot's local X axis, or 1 for a layer of type 2

`sizeZ` int

The number of tiles along the lot's local Z axis, or 1 for a layer of type 2

Exceptions

ArgumentOutOfRangeException

`sizeX` or `sizeZ` is not 0 through 255

Properties

this[int, int]

Gets/sets the value of a tile

```
public byte this[int x, int z] { get; set; }
```

Parameters

`x` int

The tile's index along the lot's local X axis, or 0 for a layer of type 2

`z` int

The tile's index along the lot's local Z axis, or 0 for a layer of type 2

Property Value

byte

Exceptions

ArgumentOutOfRangeException

A coordinate is outside the level

Level

Gets/sets the level, which cannot be −128

```
public sbyte Level { get; set; }
```

Property Value

sbyte

Remarks

−4 through 4 in the lots examined, from LowestLevel through one above HighestLevel. The section ends each layer's list of levels with −128.

Exceptions

ArgumentOutOfRangeException

The level is −128

SizeX

Gets the number of values along the lot's local X axis

```
public int SizeX { get; }
```

Property Value

int

SizeZ

Gets the number of values along the lot's local Z axis

```
public int SizeZ { get; }
```

Property Value

int

Class FloorFeaturesSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the FloorFeatures section: the floor feature layers (🔓)

```
public sealed class FloorFeaturesSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← FloorFeaturesSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game's floor feature manager reads and writes the section and exposes it to Python as `get_floor_feature`, `set_floor_feature`, `has_floor_feature`, `list_floor_features` and `scan_floor_features`. Each layer is stored as its type, the byte length of its payload and the payload; the game skips the payload of a layer type above 2, which this library does not support: a section holding one is kept undecoded.

Versions 0 and 1 are supported, as the game's reader accepts, and the game writes version 1, the only one in the lots examined. The section needs the lot's dimensions.

Constructors

FloorFeaturesSection()

Initializes a new instance of the FloorFeaturesSection class

```
public FloorFeaturesSection()
```

Properties

Layers

Gets the layers

```
public Collection<FloorFeatureLayer> Layers { get; }
```

Property Value

Collection <FloorFeatureLayer>

Remarks

The lots examined hold layer type 0 alone, types 0 and 1, or types 0, 1 and 2, in that order.

Version

Gets/sets the version of the section

```
public uint Version { get; set; }
```

Property Value

uint

Exceptions

ArgumentOutOfRangeException

The version is not 0 or 1

Class FloorIdGridSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one of the seven floor id grids: the FloorKeysSection id painted on each quarter of each tile (🔓)

```
public sealed class FloorIdGridSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← FloorIdGridSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The grid has SizeX × SizeZ tiles per layer and four ids per tile, one per FloorQuarter. Each id is a Id of the lot's FloorKeysSection, whose key is a CTProductFloorPattern (0xB4F762C9); 0 means no floor.

The types Floors, Ceilings, PoolSurfaces, SecondaryFloors and CeilingPaints have one layer per level of the lot (HasLevels), indexed by level − LowestLevel, so ground level is index −LowestLevel. TerrainFloors and PondBottoms have a single layer, for ground level. When the lot's Version is 8 or less, PoolSurfaces also has a single layer, for ground level.

Roles proven by the game's code and in-game tests: Floors is the floor pattern of each level, TerrainFloors the floor pattern painted on bare terrain, PondBottoms the pond bottom pattern and CeilingPaints the ceiling paint. In-game tests show that building a room fills Ceilings at the level above the room over exactly the room's tiles (and painting the ceiling changes only CeilingPaints), and that building a pool fills PoolSurfaces over exactly the pool's tiles at the level above the pool's block; the game's code assigns both grids to the room on the level below their layer. In-game tests show that building a room or a platform fills SecondaryFloors over its tiles with the lot's default floor key, and that painting a floor or a platform's top changes only Floors; the role of SecondaryFloors, which covers the same quarters as Floors, is still unknown.

The layers are stored one after another, each as SizeX rows of SizeZ tiles of four little-endian 16-bit ids; the section has no version.

Constructors

FloorIdGridSection(LotArchitectureSectionType, int, int, int)

Initializes a new instance of the FloorIdGridSection class with every id 0

```
public FloorIdGridSection(LotArchitectureSectionType type, int sizeX, int sizeZ,
int levelCount)
```

Parameters

`type` LotArchitectureSectionType

The type of the section, which must be one of the floor id grid types

`sizeX` int

The number of tiles along the lot's local X axis

`sizeZ` int

The number of tiles along the lot's local Z axis

`levelCount` int

The number of layers: the lot's level count, or 1 for a grid with a single layer

Exceptions

ArgumentException

`type` is not a floor id grid type

ArgumentOutOfRangeException

`sizeX`, `sizeZ` or `levelCount` is not 0 through 255, or `levelCount` is not 1 for a type with no levels

Properties

HasLevels

Gets whether the grid has one layer per level of the lot, which all types but TerrainFloors and Pond Bottoms do

```
public bool HasLevels { get; }
```

Property Value

bool

Remarks

A PoolSurfaces grid of a lot whose Version is 8 or less has levels but only one layer, for ground level.

this[int, int, int, FloorQuarter]

Gets/sets the floor key id of a quarter of a tile

```
public ushort this[int x, int z, int levelIndex, FloorQuarter quarter] { get; set; }
```

Parameters

`x` int

The tile's index along the lot's local X axis

`z` int

The tile's index along the lot's local Z axis

`levelIndex` int

The layer: the level − LowestLevel, or 0 for a grid with a single layer

`quarter` FloorQuarter

The quarter of the tile

Property Value

ushort

Exceptions

ArgumentOutOfRangeException

A coordinate is outside the grid, or `quarter` is not a FloorQuarter

LevelCount

Gets the number of layers of the grid

```
public int LevelCount { get; }
```

Property Value

int

SizeX

Gets the number of tiles of the grid along the lot's local X axis

```
public int SizeX { get; }
```

Property Value

int

SizeZ

Gets the number of tiles of the grid along the lot's local Z axis

```
public int SizeZ { get; }
```

Property Value

int

Class FloorKeyEntry

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of the FloorKeysSection: a floor pattern key and the id the floor id grids use for it

```
public sealed class FloorKeyEntry
```

Inheritance

object ← FloorKeyEntry

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Id

Gets/sets the id the FloorIdGridSection grids store for Key

```
public uint Id { get; set; }
```

Property Value

uint

Remarks

Ids are unique within the section and never 0; the game allocates the smallest unused id of 1 or more.

Key

Gets/sets the key, a CTProductFloorPattern (0xB4F762C9)

```
public ResourceKey Key { get; set; }
```

Property Value

ResourceKey

Remarks

It is the null key in 4 of the entries examined.

ReferenceCount

Gets/sets the number of references to the entry, counted in tile quarters

```
public uint ReferenceCount { get; set; }
```

Property Value

uint

Remarks

The game decrements it as quarters stop using the id and frees the entry at 0; it is never 0 in the lots examined. It equals the number of quarters holding Id across the seven FloorIdGridSection grids in 18,597 of the 18,773 entries examined in lots whose grids can be read, and is larger in the rest, which suggests other sections also hold references. This library does not recompute it.

Class FloorKeysSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the FloorKeys section: the floor pattern keys the floor id grids refer to by id (🔓)

```
public sealed class FloorKeysSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← FloorKeysSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Every nonzero id in a FloorIdGridSection is the Id of one of Entries. The game returns DefaultKey for id 0 or an id with no entry, allocates a new entry the smallest unused id of 1 or more, and frees an entry when its ReferenceCount drops to 0.

The layout follows the game's reader and writer. The section has no version.

Constructors

FloorKeysSection()

Initializes a new instance of the FloorKeysSection class

```
public FloorKeysSection()
```

Properties

DefaultKey

Gets/sets the key the game returns for id 0 and for ids with no entry

```
public ResourceKey DefaultKey { get; set; }
```

Property Value

ResourceKey

Remarks

It is the null key in every lot examined.

Entries

Gets the entries

```
public Collection<FloorKeyEntry> Entries { get; }
```

Property Value

Collection <FloorKeyEntry>

Enum FloorQuarter

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

The four triangular quarters of a tile, named by the side of the tile each one touches, in the order the game stores them

```
[SuppressMessage("Design", "CA1028: Enum Storage should be Int32", Justification = "A
quarter is stored as a 2-bit index.")]
public enum FloorQuarter : byte
```

Fields

`NegativeX = 3`

The quarter on the tile's side toward negative X

`NegativeZ = 0`

The quarter on the tile's side toward negative Z

`PositiveX = 1`

The quarter on the tile's side toward positive X

`PositiveZ = 2`

The quarter on the tile's side toward positive Z

Remarks

The game places each quarter at a centroid offset of a third of a tile from the tile's center toward its side.

Class FloorRotationGridSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one of the five floor rotation grids: the rotation of the pattern on each quarter of each tile of a FloorIdGridSection (🔓)

```
public sealed class FloorRotationGridSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← FloorRotationGridSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The grid has SizeX × SizeZ tiles per layer and one rotation of 0 through 3 per FloorQuarter. A rotation is the number of 90° clockwise turns of the pattern, as seen in game: rotating one floor tile by 90° changed all four of its quarters from 0 to 1, and so did turning a terrain floor tile 90° clockwise (in-game tests). Each grid is paired with a floor id grid: FloorRotations with Floors, CeilingRotations with Ceilings, SecondaryFloorRotations with SecondaryFloors, CeilingPaintRotations with CeilingPaints and TerrainFloorRotations with TerrainFloors; FloorIdGridSection describes the roles of the id grids and the in-game tests and code that show them.

All types but TerrainFloorRotations have one layer per level of the lot (HasLevels), indexed by level − LowestLevel; TerrainFloorRotations has a single layer, for ground level.

The layers are stored one after another, each as SizeX rows of SizeZ bytes; each byte holds the four rotations of a tile, two bits each, the NegativeZ quarter in the lowest bits, as the game's getter and setter read and write them. The section has no version.

Constructors

FloorRotationGridSection(LotArchitectureSectionType, int, int, int)

Initializes a new instance of the FloorRotationGridSection class with every rotation 0

```
public FloorRotationGridSection(LotArchitectureSectionType type, int sizeX, int sizeZ,
int levelCount)
```

Parameters

`type` LotArchitectureSectionType

The type of the section, which must be one of the floor rotation grid types

`sizeX` int

The number of tiles along the lot's local X axis

`sizeZ` int

The number of tiles along the lot's local Z axis

`levelCount` int

The number of layers: the lot's level count, or 1 for a grid with a single layer

Exceptions

ArgumentException

`type` is not a floor rotation grid type

ArgumentOutOfRangeException

`sizeX`, `sizeZ` or `levelCount` is not 0 through 255, or `levelCount` is not 1 for a type with no levels

Properties

HasLevels

Gets whether the grid has one layer per level of the lot, which all types but TerrainFloorRotations do

```
public bool HasLevels { get; }
```

Property Value

bool

this[int, int, int, FloorQuarter]

Gets/sets the rotation of the pattern on a quarter of a tile, as the number of 90° clockwise turns

```
public byte this[int x, int z, int levelIndex, FloorQuarter quarter] { get; set; }
```

Parameters

`x` int

The tile's index along the lot's local X axis

`z` int

The tile's index along the lot's local Z axis

`levelIndex` int

The layer: the level − LowestLevel, or 0 for a grid with a single layer

`quarter` FloorQuarter

The quarter of the tile

Property Value

byte

Exceptions

ArgumentOutOfRangeException

A coordinate is outside the grid, `quarter` is not a FloorQuarter, or the rotation being set is not 0 through 3

LevelCount

Gets the number of layers of the grid

```
public int LevelCount { get; }
```

Property Value

int

SizeX

Gets the number of tiles of the grid along the lot's local X axis

```
public int SizeX { get; }
```

Property Value

int

SizeZ

Gets the number of tiles of the grid along the lot's local Z axis

```
public int SizeZ { get; }
```

Property Value

int

Class LegacyFencePoint

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of the LegacyFencePointsSection

```
public sealed class LegacyFencePoint
```

Inheritance

object ← LegacyFencePoint

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Unknown1, Unknown2, Unknown3, Key (type, group, instance).

Properties

Key

Gets/sets the catalog key of the entry

```
public ResourceKey Key { get; set; }
```

Property Value

ResourceKey

Remarks

A fence (0x0418FE2A) key in every Points1 entry and a ceiling rail (CTProductCeilingRail 0x3F0C529A) key in every Points2 entry examined, group 0.

Unknown1

Gets/sets the first value of the entry

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Remarks

Hypothesis: a lot level. 0, 1, 2 or 4 in the lots examined; bits 28 through 31 of the LegacyWallGraph Section codes, the levels under the same hypothesis, are 0 through 4.

Unknown2

Gets/sets the second value of the entry

```
public float Unknown2 { get; set; }
```

Property Value

float

Remarks

A whole number in the lots examined. Hypothesis: a coordinate. In each of the 252 entries examined in lots that also hold a LegacyWallGraphSection, it, Unknown3 and Unknown1 equal bits 7 through 13, bits 0 through 6 and bits 28 through 31 of a Code or Code of that section (or bits 21 through 27, bits 14 through 20 and bits 28 through 31); the other 80 entries are in lots without that section.

Unknown3

Gets/sets the third value of the entry

```
public float Unknown3 { get; set; }
```

Property Value

float

Remarks

A whole number in the lots examined. Hypothesis: a coordinate.

Class LegacyFencePointsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the LegacyFencePoints section: two lists of points with fence and ceiling rail keys, found only in older world files (🔓)

```
public sealed class LegacyFencePointsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← LegacyFencePointsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

No code in the game's simulation library reads or writes this section (other parts of the game were not examined), so the layout comes from the lots examined and every meaning is a hypothesis. The lots examined hold 21 of these sections, all in world files, 15 of them empty. Hypothesis: the points are post or end-piece positions of the fences and ceiling rails of the LegacyWallGraphSection.

Version 2 is the only one supported, and the only one in the lots examined.

Layout: the version (uint), an int count and the Points1, then an int count and the Points2.

Constructors

LegacyFencePointsSection()

Initializes a new instance of the LegacyFencePointsSection class

```
public LegacyFencePointsSection()
```

Properties

Points1

Gets the first list of points

```
public Collection<LegacyFencePoint> Points1 { get; }
```

Property Value

Collection<LegacyFencePoint>

Remarks

Their keys are fence keys in every lot examined.

Points2

Gets the second list of points

```
public Collection<LegacyFencePoint> Points2 { get; }
```

Property Value

Collection<LegacyFencePoint>

Remarks

Their keys are ceiling rail keys in every lot examined.

Class LegacyWallGraphEdge

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an edge of Edges

```
public sealed class LegacyWallGraphEdge
```

Inheritance

object ← LegacyWallGraphEdge

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Code, an int count, then the Parts.

Properties

Code

Gets/sets the code of the edge

```
public uint Code { get; set; }
```

Property Value

uint

Remarks

Hypothesis: the edge's two endpoints and level, as 7-bit coordinates in bits 0 through 6, 7 through 13, 14 through 20 and 21 through 27 and the level in bits 28 through 31. Bits 6, 13, 20, 27 and 31 are never set in the lots examined, and read that way every edge lies within the lot and joins neighboring points: 2,784 of the 2,824 codes of the lots examined (counting those of the product edges) differ by 1 in one coordinate, the other 40 by 1 in both.

Parts

Gets the parts of the edge

```
public Collection<LegacyWallGraphEdgePart> Parts { get; }
```

Property Value

Collection <LegacyWallGraphEdgePart>

Remarks

1 or 2 in the lots examined.

Class LegacyWallGraphEdgePart

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a part of a LegacyWallGraphEdge

```
public sealed class LegacyWallGraphEdgePart
```

Inheritance

object ← LegacyWallGraphEdgePart

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Unknown1 through Unknown4, WallPattern1Id, WallPattern2Id, ProductId.

Properties

ProductId

Gets/sets the Id of the part's product in Products

```
public uint ProductId { get; set; }
```

Property Value

uint

Remarks

It refers to a CTProductWall (0x9151E6BC) key in every lot examined.

Unknown1

Gets/sets the first ushort of the part

```
public ushort Unknown1 { get; set; }
```

Property Value

ushort

Remarks

0, 1 or 3 in the lots examined.

Unknown2

Gets/sets the second ushort of the part

```
public ushort Unknown2 { get; set; }
```

Property Value

ushort

Remarks

0 through 43 in the lots examined.

Unknown3

Gets/sets the third ushort of the part

```
public ushort Unknown3 { get; set; }
```

Property Value

ushort

Remarks

Equal to Unknown4 in 1,374 of the 1,590 parts examined, where both are 76, 204, 384, 512, 545, 546 or 640; 0 through 546 otherwise.

Unknown4

Gets/sets the fourth ushort of the part

```
public ushort Unknown4 { get; set; }
```

Property Value

ushort

WallPattern1Id

Gets/sets the Id of one wall pattern in WallPatterns

```
public uint WallPattern1Id { get; set; }
```

Property Value

uint

Remarks

Hypothesis: the patterns of the two sides of the wall.

WallPattern2Id

Gets/sets the Id of the other wall pattern in WallPatterns

```
public uint WallPattern2Id { get; set; }
```

Property Value

uint

Class LegacyWallGraphProductEdge

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an edge of ProductEdges1 or ProductEdges2

```
public sealed class LegacyWallGraphProductEdge
```

Inheritance

object ← LegacyWallGraphProductEdge

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Code, Unknown1 through Unknown5, ProductId.

Properties

Code

Gets/sets the code of the edge

```
public uint Code { get; set; }
```

Property Value

uint

Remarks

Packed like Code (hypothesis).

ProductId

Gets/sets the Id of the edge's product in Products

```
public uint ProductId { get; set; }
```

Property Value

uint

Unknown1

Gets/sets the first uint after Code

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Remarks

1 in every lot examined.

Unknown2

Gets/sets the second uint after Code

```
public uint Unknown2 { get; set; }
```

Property Value

uint

Remarks

0 in every lot examined.

Unknown3

Gets/sets the third uint after Code

```
public uint Unknown3 { get; set; }
```

Property Value

uint

Remarks

0xCCCCCCCC in every lot examined, the fill pattern of uninitialized memory in debug builds of the compiler the game uses; hypothesis: not meaningful.

Unknown4

Gets/sets the fourth uint after Code

```
public uint Unknown4 { get; set; }
```

Property Value

uint

Remarks

0xCCCCCCCC in every lot examined, like Unknown3.

Unknown5

Gets/sets the fifth uint after Code

```
public uint Unknown5 { get; set; }
```

Property Value

uint

Remarks

0xCCCCCCCC in every lot examined, like Unknown3.

Class LegacyWallGraphSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the LegacyWallGraph section: a graph of wall edges with their wall patterns and products, found only in older world files (🔓)

```
public sealed class LegacyWallGraphSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← LegacyWallGraphSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

No code in the game's simulation library reads or writes this section (other parts of the game were not examined), so the layout comes from the lots examined and every meaning is a hypothesis. The lots examined hold 21 of these sections, all in world files; 17 are empty. The two tables have the shape of the game's ProductTable, and in each of the four non-empty sections examined every id is referenced and every ReferenceCount equals the number of references in the section.

Version 1 is the only one supported, and the only one in the lots examined.

Layout: the version (uint), the Products table, the WallPatterns table, then three lists, each an int count and its entries: Edges, ProductEdges1 and ProductEdges2.

Constructors

LegacyWallGraphSection()

Initializes a new instance of the LegacyWallGraphSection class

```
public LegacyWallGraphSection()
```

Properties

Edges

Gets the edges with wall parts

```
public Collection<LegacyWallGraphEdge> Edges { get; }
```

Property Value

Collection<LegacyWallGraphEdge>

ProductEdges1

Gets the first list of edges with a product

```
public Collection<LegacyWallGraphProductEdge> ProductEdges1 { get; }
```

Property Value

Collection<LegacyWallGraphProductEdge>

Remarks

Their products are ceiling rails (CTProductCeilingRail 0x3F0C529A) in every lot examined.

ProductEdges2

Gets the second list of edges with a product

```
public Collection<LegacyWallGraphProductEdge> ProductEdges2 { get; }
```

Property Value

Collection<LegacyWallGraphProductEdge>

Remarks

Their products are fences (0x0418FE2A) in every lot examined.

Products

Gets the table of the products the edges refer to

```
public ProductTable Products { get; }
```

Property Value

ProductTable

Remarks

In the lots examined it holds fence (0x0418FE2A), CTProductWall (0x9151E6BC) and ceiling rail (CTProductCeilingRail 0x3F0C529A) keys, and its DefaultKey is the null key.

WallPatterns

Gets the table of the wall patterns the edge parts refer to

```
public ProductTable WallPatterns { get; }
```

Property Value

ProductTable

Remarks

In the lots examined it holds only CTProductWallPattern (0xD5F0F921) keys, and its DefaultKey is the null key.

Class LotArchitecture

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the architecture of a lot: its walls, floors, roofs, stairs, terrain and the rest of its build, as typed sections (🔓)

```
public sealed class LotArchitecture
```

Inheritance

object ← LotArchitecture

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game stores a lot's architecture as a RecordBundle in three places: the WorldLotArchitecture resources of world files, the architecture data of each lot in a save game (see ZoneArchitecture) and tray blueprints (see Blueprint). Each record of the bundle is one section, keyed by its LotArchitectureSectionType, group 0 and instance 1.

Many sections can only be read with the lot's dimensions, which the LotSize section gives, so the bundle, not the section, is the unit of decoding. Records Decode(RecordBundle) cannot turn into sections are kept, unchanged, in UndecodedRecords: those of types this library does not know, those with another group or instance, those with a version (of the section or of an element of it) or a floor feature layer type this library does not support, and those that need dimensions an older LotSize section does not give (the game itself discards those).

Encode() writes the sections in the layout of their versions, so decoding and encoding the architecture of any lot the game wrote reproduces it byte for byte.

Where the documentation of the sections speaks of the lots, sections or entries examined, it means the 2,919 lot architectures examined while developing this library, at the game's patch level as of October 2026: those of the lots of the game's installed world files (of which twelve base-game worlds had been replaced by a community lot-replacement mod), the 179 lots of the Maxis originals of those twelve worlds, the lots of four save games and 73 tray blueprints.

Fields

SectionInstance

Gets the instance of the record of every section

```
public const ulong SectionInstance = 1
```

Field Value

ulong

Properties

CeilingPaintRotations

Gets/sets the rotation of the ceiling paint of each level (🔓)

```
public FloorRotationGridSection? CeilingPaintRotations { get; set; }
```

Property Value

FloorRotationGridSection

CeilingPaints

Gets/sets the ceiling paint of each level (🔓)

```
public FloorIdGridSection? CeilingPaints { get; set; }
```

Property Value

FloorIdGridSection

CeilingRotations

Gets/sets the rotation of the ceiling of each level (🔓)

```
public FloorRotationGridSection? CeilingRotations { get; set; }
```

Property Value

FloorRotationGridSection

Remarks

It is paired with Ceilings, whose role in-game tests and the game's code show.

Ceilings

Gets/sets the ceiling of each level (🔓)

```
public FloorIdGridSection? Ceilings { get; set; }
```

Property Value

FloorIdGridSection

Remarks

In-game tests show that building a room fills this grid at the level above the room over exactly the room's tiles, and that painting the ceiling changes only CeilingPaints; the game's code assigns the grid to the room on the level below.

FloorFeatures

Gets/sets the floor feature layers (🔓)

```
public FloorFeaturesSection? FloorFeatures { get; set; }
```

Property Value

FloorFeaturesSection

FloorKeys

Gets/sets the floor keys the floor id grids refer to (🔓)

```
public FloorKeysSection? FloorKeys { get; set; }
```

Property Value

FloorKeysSection

FloorRotations

Gets/sets the rotation of the floor pattern of each level (🔓)

```
public FloorRotationGridSection? FloorRotations { get; set; }
```

Property Value

FloorRotationGridSection

Floors

Gets/sets the floor pattern of each level (🔓)

```
public FloorIdGridSection? Floors { get; set; }
```

Property Value

FloorIdGridSection

LegacyFencePoints

Gets/sets the points with fence and ceiling rail keys of older world files (🔓)

```
public LegacyFencePointsSection? LegacyFencePoints { get; set; }
```

Property Value

LegacyFencePointsSection

Remarks

The name reflects the catalog key types the data holds; the section appears only in older world files.

LegacyWallGraph

Gets/sets the graph of edges with wall pattern and product keys of older world files (🔓)

```
public LegacyWallGraphSection? LegacyWallGraph { get; set; }
```

Property Value

LegacyWallGraphSection

Remarks

The name reflects the catalog key types the data holds; the section appears only in older world files.

LotInfo

Gets/sets the lot's levels, style and creators (🔓)

```
public LotInfoSection? LotInfo { get; set; }
```

Property Value

LotInfoSection

LotObjects

Gets/sets the objects stored with the architecture (🔓)

```
public LotObjectsSection? LotObjects { get; set; }
```

Property Value

LotObjectsSection

LotSize

Gets/sets the lot's dimensions (🔓)

```
public LotSizeSection? LotSize { get; set; }
```

Property Value

LotSizeSection

ModularPieces

Gets/sets the modular pieces (🔓)

```
public ModularPiecesSection? ModularPieces { get; set; }
```

Property Value

ModularPiecesSection

ObjectFallbacks

Gets/sets the objects the game falls back to placing when others are unavailable (🔓)

```
public ObjectFallbacksSection? ObjectFallbacks { get; set; }
```

Property Value

ObjectFallbacksSection

ObjectTileBounds

Gets/sets the tiles each object covers (🔓)

```
public ObjectTileBoundsSection? ObjectTileBounds { get; set; }
```

Property Value

ObjectTileBoundsSection

PendingPlacement

Gets/sets the pending placement (🔓)

```
public PendingPlacementSection? PendingPlacement { get; set; }
```

Property Value

PendingPlacementSection

Remarks

When its state is 1, the game centers a footprint of the stored width, depth and rotation on the lot and builds a placement operation with the lot's BlueprintInstance.

PlexGroups

Gets/sets the groups of values the Plexes section refers to by id (🔓)

```
public PlexGroupsSection? PlexGroups { get; set; }
```

Property Value

PlexGroupsSection

Remarks

That the values are block ids is a hypothesis.

Plexes

Gets/sets the plexes (units) of the lot (🔓)

```
public PlexesSection? Plexes { get; set; }
```

Property Value

PlexesSection

PondBottoms

Gets/sets the pond bottom pattern (🔓)

```
public FloorIdGridSection? PondBottoms { get; set; }
```

Property Value

FloorIdGridSection

Ponds

Gets/sets the ponds (🔓)

```
public PondsSection? Ponds { get; set; }
```

Property Value

PondsSection

PoolSurfaces

Gets/sets the pool surface pattern of each level (🔓)

```
public FloorIdGridSection? PoolSurfaces { get; set; }
```

Property Value

FloorIdGridSection

Remarks

In-game tests show that building a pool fills this grid over exactly the pool's tiles at the level above the pool's block; the game's code assigns the grid to the room on the level below.

Reserved1

Gets/sets the first reserved header value of the record bundle, which no code in the game's simulation library reads (other parts of the game were not examined)

```
public uint Reserved1 { get; set; }
```

Property Value

uint

Reserved2

Gets/sets the second reserved header value of the record bundle, which no code in the game's simulation library reads (other parts of the game were not examined)

```
public uint Reserved2 { get; set; }
```

Property Value

uint

ResourceManifest

Gets/sets the keys of the catalog resources the other sections reference (🔓)

```
public ResourceManifestSection? ResourceManifest { get; set; }
```

Property Value

ResourceManifestSection

Revision

Gets/sets the lot's revision counter (🔓)

```
public RevisionSection? Revision { get; set; }
```

Property Value

RevisionSection

Roofs

Gets/sets the roofs (🔓)

```
public RoofsSection? Roofs { get; set; }
```

Property Value

RoofsSection

SecondaryFloorRotations

Gets/sets the rotation of the second floor layer of each level (🔓)

```
public FloorRotationGridSection? SecondaryFloorRotations { get; set; }
```

Property Value

FloorRotationGridSection

SecondaryFloors

Gets/sets the second floor layer of each level (🔓)

```
public FloorIdGridSection? SecondaryFloors { get; set; }
```

Property Value

FloorIdGridSection

Remarks

In-game tests show that building a room or a platform fills this grid over its tiles with the lot's default floor key, and that painting a floor or a platform's top changes only Floors; the grid's role is unknown.

Sections

Gets the decoded sections, in ascending order of type

```
public IReadOnlyList<LotArchitectureSection> Sections { get; }
```

Property Value

IReadOnlyList<LotArchitectureSection>

Stairs

Gets/sets the stairs and ladders (🔓)

```
public StairsSection? Stairs { get; set; }
```

Property Value

StairsSection

TerrainFloorRotations

Gets/sets the rotation of the floor pattern painted on the terrain (🔓)

```
public FloorRotationGridSection? TerrainFloorRotations { get; set; }
```

Property Value

FloorRotationGridSection

TerrainFloors

Gets/sets the floor pattern painted on the terrain (🔓)

```
public FloorIdGridSection? TerrainFloors { get; set; }
```

Property Value

FloorIdGridSection

TerrainHeights

Gets/sets the terrain heights (🔓)

```
public TerrainHeightsSection? TerrainHeights { get; set; }
```

Property Value

TerrainHeightsSection

TerrainPaint

Gets/sets the terrain paint layers (🔓)

```
public TerrainPaintSection? TerrainPaint { get; set; }
```

Property Value

TerrainPaintSection

UndecodedRecords

Gets the records that were not decoded into sections, which Encode() writes back unchanged

```
public Collection<RecordBundleRecord> UndecodedRecords { get; }
```

Property Value

Collection <RecordBundleRecord>

Unknown7B33C1F5

Gets/sets the section of unknown purpose of type Unknown7B33C1F5 ( )

```
public Unknown7B33C1F5Section? Unknown7B33C1F5 { get; set; }
```

Property Value

Unknown7B33C1F5Section

UnknownC9106475

Gets/sets the section of unknown purpose of type UnknownC9106475 ( )

```
public UnknownC9106475Section? UnknownC9106475 { get; set; }
```

Property Value

UnknownC9106475Section

Venue

Gets/sets the lot's venue ( )

```
public VenueSection? Venue { get; set; }
```

Property Value

VenueSection

VertexGraphs

Gets/sets the graphs of vertices and edges whose version decides the layout of WallEdges (🔓)

```
public VertexGraphsSection? VertexGraphs { get; set; }
```

Property Value

VertexGraphsSection

Remarks

The game reads only the section's version; what the graphs describe is not established. Sims 4 Studio calls this section WallCutout.

WallBlocks

Gets/sets the blocks: rooms, wall lines, fences, pools and fountains (🔓)

```
public WallBlocksSection? WallBlocks { get; set; }
```

Property Value

WallBlocksSection

WallEdges

Gets/sets the wall patterns and products along wall edges (🔓)

```
public WallEdgesSection? WallEdges { get; set; }
```

Property Value

WallEdgesSection

Weather

Gets/sets the lot's seasonal weather interpolation value (🔓)

```
public WeatherSection? Weather { get; set; }
```

Property Value

WeatherSection

Methods

Decode(RecordBundle)

Decodes the architecture of a lot from a record bundle

```
public static LotArchitecture Decode(RecordBundle bundle)
```

Parameters

`bundle` RecordBundle

Returns

LotArchitecture

Exceptions

InvalidDataException

One of the bundle's sections is not well-formed

Decode(ReadOnlyMemory<byte>)

Decodes the architecture of a lot from the content of a record bundle

```
public static LotArchitecture Decode(ReadOnlyMemory<byte> bundleData)
```

Parameters

`bundleData` ReadOnlyMemory<byte>

Returns

LotArchitecture

Exceptions

InvalidDataException

The data is not a well-formed record bundle, or one of its sections is not well-formed

Encode()

Encodes the architecture as the content of a record bundle

```
public ReadOnlyMemory<byte> Encode()
```

Returns

ReadOnlyMemory<byte>

Exceptions

InvalidOperationException

A section cannot be encoded as it is, or a section and an undecoded record have the same key

GetManifestKeys()

Gets the keys the sections reference, in the order the game's saver would collect them into the Resource Manifest

```
public IReadOnlyList<ResourceKey> GetManifestKeys()
```

Returns

IReadOnlyList<ResourceKey>

Remarks

The game builds the manifest by recording every key it writes while saving the sections that precede the manifest, in its fixed save order, skipping repeats. This method does the same with the current sections, so assigning the result to a new ResourceManifestSection keeps the manifest consistent after edits. No code in the game's simulation library reads the manifest back (other parts of the game were not examined). For 2,907 of 2,912 lots in the game's world files, save games and tray blueprints examined, the result equals the stored manifest; the exceptions are lots of older world files whose blocks hold inline keys, which the current game no longer writes, and one saved lot whose stored manifest lacks two keys its sections reference.

Exceptions

InvalidOperationException

A section cannot be encoded as it is

ToRecordBundle()

Creates a record bundle holding the architecture

```
public RecordBundle ToRecordBundle()
```

Returns

RecordBundle

Exceptions

InvalidOperationException

A section cannot be encoded as it is, or a section and an undecoded record have the same key

Class LotArchitectureSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one section of a lot's architecture: one record of the lot's RecordBundle

```
public abstract class LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection

Derived

FloorFeaturesSection, FloorIdGridSection, FloorKeysSection, FloorRotationGridSection, LegacyFencePointsSection, LegacyWallGraphSection, LotInfoSection, LotObjectsSection, LotSizeSection, ModularPiecesSection, ObjectFallbacksSection, ObjectTileBoundsSection, PendingPlacementSection, PlexGroupsSection, PlexesSection, PondsSection, ResourceManifestSection, RevisionSection, RoofsSection, StairsSection, TerrainHeightsSection, TerrainPaintSection, Unknown7B33C1F5Section, UnknownC9106475Section, VenueSection, VertexGraphsSection, WallBlocksSection, WallEdgesSection, WeatherSection

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Type

Gets the type of the section

```
public LotArchitectureSectionType Type { get; }
```

Property Value

LotArchitectureSectionType

Enum LotArchitectureSectionType

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

The types of the sections of a lot's architecture, each of which is one record of the lot's RecordBundle

```
[SuppressMessage("Design", "CA1008: Enums should have zero value", Justification = "No
section has type zero.")]
[SuppressMessage("Design", "CA1028: Enum Storage should be Int32", Justification = "Section
types are resource types, which are 32-bit unsigned.")]
public enum LotArchitectureSectionType : uint
```

Fields

`CeilingPaintRotations = 3534689615`

The rotation of the ceiling paint of each level (FloorRotationGridSection)

`CeilingPaints = 2427045927`

The ceiling paint of each level, as floor key ids (FloorIdGridSection)

`CeilingRotations = 1703255730`

The rotation of the ceiling of each level (FloorRotationGridSection)

`Ceilings = 501723820`

The ceiling of each level, as floor key ids (FloorIdGridSection)

In-game tests show that building a room fills this grid at the level above the room over exactly the room's tiles, and that painting the ceiling changes only CeilingPaints; the game's code assigns the grid to the room on the level below.

`FloorFeatures = 4062213290`

The floor feature layers (FloorFeaturesSection)

`FloorKeys = 4172807529`

The floor keys the floor id grids refer to (FloorKeysSection)

`FloorRotations = 1123611834`

The rotation of the floor pattern of each level (FloorRotationGridSection)

`Floors = 622686172`

The floor pattern of each level, as floor key ids (FloorIdGridSection)

`LegacyFencePoints = 1980314405`

Points with fence and ceiling rail keys (LegacyFencePointsSection)

The name reflects the catalog key types the data holds; the section appears only in older world files.

`LegacyWallGraph = 2465065149`

A graph of edges with wall pattern and product keys (LegacyWallGraphSection)

The name reflects the catalog key types the data holds; the section appears only in older world files.

`LotInfo = 545371366`

The lot's levels, style and creators (LotInfoSection)

`LotObjects = 3325813684`

Objects stored with the architecture (LotObjectsSection)

`LotSize = 1156008438`

The lot's dimensions (LotSizeSection)

`ModularPieces = 4208247409`

The modular pieces (ModularPiecesSection)

`ObjectFallbacks = 1042161`

The objects the game falls back to placing when others are unavailable (ObjectFallbacksSection)

`ObjectTileBounds = 2282844662`

The tiles each object covers (ObjectTileBoundsSection)

`PendingPlacement = 2482231023`

A pending placement (PendingPlacementSection)

When its state is 1, the game centers a footprint of the stored width, depth and rotation on the lot and builds a placement operation with the lot's BlueprintInstance.

`PlexGroups = 1914205540`

Groups of values the Plexes section refers to by id (PlexGroupsSection)

That the values are block ids is a hypothesis.

`Plexes = 2434334207`

The plexes (units) of the lot (PlexesSection)

`PondBottoms = 2725680907`

The pond bottom pattern, as floor key ids (FloorIdGridSection)

`Ponds = 2188151641`

The ponds (PondsSection)

`PoolSurfaces = 1040855313`

The pool surface pattern of each level, as floor key ids (FloorIdGridSection)

In-game tests show that building a pool fills this grid over exactly the pool's tiles at the level above the pool's block; the game's code assigns the grid to the room on the level below.

`ResourceManifest = 152626801`

The keys of the catalog resources the other sections reference (ResourceManifestSection)

`Revision = 2794582592`

The lot's revision counter (RevisionSection)

`Roofs = 3285814088`

The roofs (RoofsSection)

`SecondaryFloorRotations = 3862978298`

The rotation of the second floor layer of each level (FloorRotationGridSection)

`SecondaryFloors = 1704918004`

A second floor layer of each level, as floor key ids (FloorIdGridSection)

In-game tests show that building a room or a platform fills this grid over its tiles with the lot's default floor key, and that painting a floor or a platform's top changes only Floors; the grid's role is unknown.

`Stairs = 186155609`

The stairs and ladders (StairsSection)

`TerrainFloorRotations = 3315290771`

The rotation of the floor pattern painted on the terrain (FloorRotationGridSection)

`TerrainFloors = 2874388677`

The floor pattern painted on the terrain, as floor key ids (FloorIdGridSection)

`TerrainHeights = 3782893869`

The terrain heights (TerrainHeightsSection)

`TerrainPaint = 756498313`

The terrain paint layers (TerrainPaintSection)

`Unknown7B33C1F5 = 2066989557`

A section of unknown purpose found in three lots of world files (Unknown7B33C1F5Section)

`UnknownC9106475 = 3373294709`

A section of unknown purpose (UnknownC9106475Section)

`Venue = 3871070174`

The lot's venue (VenueSection)

`VertexGraphs = 2067194067`

Graphs of vertices and edges whose version decides the layout of the WallEdges section (VertexGraphs Section)

The game reads only the section's version; what the graphs describe is not established. Sims 4 Studio calls this section WallCutout.

`WallBlocks = 2425094600`

The blocks: rooms, wall lines, fences, pools and fountains (WallBlocksSection)

`WallEdges = 1774819204`

The wall patterns and products along wall edges (WallEdgesSection)

`Weather = 1236347683`

The lot's seasonal weather interpolation value (WeatherSection)

Remarks

The game keys every section by its type, group 0 and instance 1. Names reflect what the game's code or in-game tests show each section holds; where neither does, the name is the type's hexadecimal value.

Class LotInfoPlexValue

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one entry of the plex map of a LotInfoSection: an optional value per plex unit

```
public sealed class LotInfoPlexValue
```

Inheritance

object ← LotInfoPlexValue

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game hands the map to its plex manager after loading. In-game tests show that the value grows with each paid build change (by 960 for a room, 64 for a floor fill, 64 for a ceiling, 16 for four ground tiles, 4 for one terrain floor tile, 75 for three fence segments) and is unchanged by a rotation or a quarter-tile repaint; that it is the architecture's value is a hypothesis.

Constructors

LotInfoPlexValue()

Initializes a new instance of the LotInfoPlexValue class

```
public LotInfoPlexValue()
```

Properties

HasValue

Gets/sets whether the entry has a value

```
public bool HasValue { get; set; }
```

Property Value

bool

Remarks

The game writes Value even when this is false. It is true in all but five of the entries examined.

UnitId

Gets/sets the id of the plex unit the entry belongs to

```
public uint UnitId { get; set; }
```

Property Value

uint

Remarks

Stored from version 15 on; versions 12 through 14 hold a single entry for unit 0.

Value

Gets/sets the value of the entry

```
public uint Value { get; set; }
```

Property Value

uint

Class LotInfoSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the LotInfo section: the lot's levels, style, blueprint and users (🔓)

```
public sealed class LotInfoSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← LotInfoSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout is Unknown1 (from version 13 on), the lowest and highest levels (signed bytes), one height per level from the lowest to the highest, LevelElevationOffset (from version 6 on), NegativeLevelElevationOffset (from version 16 on), Unknown2 (from version 17 on), Unknown3 (from version 18 on) and StyleKey (from version 7 on). From version 11 on, BlueprintInstance, the ids of the two users, Uuid, the names of the two users and, from version 19 on, the rest of the two user records follow. Versions 12 through 14 end with one plex map entry (value and has-value flag) and versions 15 and later with a 32-bit entry count and the entries.

The highest level is not kept: it is LowestLevel plus the number of LevelHeights minus 1, which the game's reader relies on to read the heights. The number of heights equals LevelCount.

Versions 5 through 20 are supported, and the game writes version 20. The game reads some other sections only for some versions of this one (for example PendingPlacementSection from version 9 on).

Constructors

LotInfoSection()

Initializes a new instance of the LotInfoSection class

```
public LotInfoSection()
```

Properties

BlueprintInstance

Gets/sets the tray instance of the blueprint the lot was saved as

```
public ulong BlueprintInstance { get; set; }
```

Property Value

ulong

Remarks

Present from version 11 on. In-game tests show that it equals the tray instance of each blueprint saved, and it is nonzero in every blueprint examined; the game passes it to the placement operation it builds from the PendingPlacementSection.

FirstUser

Gets the first user record

```
public LotInfoUser FirstUser { get; }
```

Property Value

LotInfoUser

Remarks

Present from version 11 on.

HighestLevel

Gets the highest level of the lot: LowestLevel plus the number of LevelHeights minus 1

```
public int HighestLevel { get; }
```

Property Value

int

LevelElevationOffset

Gets/sets the vertical offset the game adds to the elevation of levels 0 and up

```
public float LevelElevationOffset { get; set; }
```

Property Value

float

Remarks

Present from version 6 on. The game sets it together with NegativeLevelElevationOffset, quantized, moving the lot's blocks with it; that it is the foundation height is a hypothesis.

LevelHeights

Gets the height of each level, from the lowest to the highest

```
public Collection<float> LevelHeights { get; }
```

Property Value

Collection<float>

Remarks

The game sums these to get the elevation of a level. They are 3, 4 or 5 in the lots examined. Raising a level's walls from 3 m to 5 m in game changed that level's height from 3 to 5, and the base elevations of the blocks, roofs and stair tops above it followed (in-game test).

LowestLevel

Gets/sets the lowest level of the lot

```
public sbyte LowestLevel { get; set; }
```

Property Value

sbyte

NegativeLevelElevationOffset

Gets/sets the vertical offset the game adds to the elevation of levels below 0

```
public float NegativeLevelElevationOffset { get; set; }
```

Property Value

float

Remarks

Present from version 16 on.

PlexValues

Gets the entries of the plex map

```
public Collection<LotInfoPlexValue> PlexValues { get; }
```

Property Value

Collection<LotInfoPlexValue>

Remarks

Versions before 12 hold none, and versions 12 through 14 exactly one.

SecondUser

Gets the second user record

```
public LotInfoUser SecondUser { get; }
```

Property Value

LotInfoUser

Remarks

Present from version 11 on.

StyleKey

Gets/sets the key of the lot's style, a `CTProductStyle` (0x9F5CFF10) or the null key

```
public ResourceKey StyleKey { get; set; }
```

Property Value

ResourceKey

Remarks

Present from version 7 on. The game replaces a missing or invalid style with the default style while loading.

Unknown1

Gets/sets the first unknown value

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Remarks

Present from version 13 on. It defaults to 1 in the game and is 1 in all but 24 lots examined (which hold 0).

Unknown2

Gets/sets the second unknown value

```
public bool Unknown2 { get; set; }
```

Property Value

bool

Remarks

Present from version 17 on. The game overwrites it while loading with whether the lot has plexes in a certain state or another flag of the lot is set.

Unknown3

Gets/sets the third unknown value

```
public bool Unknown3 { get; set; }
```

Property Value

bool

Remarks

Present from version 18 on; the game sets it for older versions. When it is set, the game moves the lot's objects to their level's height minus a constant while loading, then clears it.

Uuid

Gets/sets the bytes stored between the user ids and the user names

```
public ReadOnlyMemory<byte> Uuid { get; set; }
```

Property Value

ReadOnlyMemory<byte>

Remarks

Present from version 11 on. It is empty or a 16-byte RFC 4122 UUID (version 1 or 4) in the lots examined; its purpose is unknown.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 5 through 20

Class LotInfoUser

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one of the two user records of a LotInfoSection

```
public sealed class LotInfoUser
```

Inheritance

object ← LotInfoUser

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The names are account names in the lots examined (or empty); that the first record is the lot's creator and the second its last modifier is a hypothesis.

The section stores both Id values and both Name values first; from version 19 on, each record's Unknown1, Unknown2 and RepeatedId follow.

Constructors

LotInfoUser()

Initializes a new instance of the LotInfoUser class

```
public LotInfoUser()
```

Properties

Id

Gets/sets the id of the user

```
public ulong Id { get; set; }
```

Property Value

ulong

Remarks

That it is an account id is a hypothesis.

Name

Gets/sets the name of the user

```
public string Name { get; set; }
```

Property Value

string

RepeatedId

Gets/sets the id that repeats Id

```
public ulong RepeatedId { get; set; }
```

Property Value

ulong

Remarks

Present from version 19 on. The game reads it after Id and keeps it in place of Id; the two are equal in every lot examined.

Unknown1

Gets/sets the first unknown value

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Remarks

Present from version 19 on. It is 0 or 100 in the lots examined.

Unknown2

Gets/sets the second unknown value

```
public string Unknown2 { get; set; }
```

Property Value

string

Remarks

Present from version 19 on. It is empty or an account name in the lots examined.

Class LotObjectsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the LotObjects section: the objects of the lot, as an `EA.Sims4.Network.LotObjectList` message (🔓)

```
public sealed class LotObjectsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← LotObjectsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Version 1 is the only one; the game fails to load the lot with any other. The version is followed by the 32-bit length of the serialized message. The game writes the section only when the list is not empty.

Re-serializing the message reproduces every payload examined byte for byte.

Constructors

LotObjectsSection()

Initializes a new instance of the LotObjectsSection class

```
public LotObjectsSection()
```

Properties

Objects

Gets/sets the message

```
public LotObjectList Objects { get; set; }
```

Property Value

LotObjectList

Exceptions

ArgumentNullException

The value is null

Class LotSizeSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the LotSize section: the lot's dimensions (🔓)

```
public sealed class LotSizeSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← LotSizeSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The grids of the other sections are SizeX tiles along the lot's local X axis by SizeZ tiles along its local Z axis, with one layer per level. LevelCount is the number of levels from the lowest to the highest of LotInfoSection, inclusive.

Versions 2 through 13 are supported, and the game writes version 13. Version 2 holds no dimensions (HasDimensions is false); the game discards the floor data of such a lot.

Constructors

LotSizeSection()

Initializes a new instance of the LotSizeSection class

```
public LotSizeSection()
```

Properties

HasDimensions

Gets whether the section holds the lot's dimensions, which it does from version 3 on

```
public bool HasDimensions { get; }
```

Property Value

bool

LevelCount

Gets/sets the number of levels of the lot, from the lowest to the highest, inclusive

```
public byte LevelCount { get; set; }
```

Property Value

byte

SizeX

Gets/sets the number of tiles of the lot along its local X axis

```
public byte SizeX { get; set; }
```

Property Value

byte

SizeZ

Gets/sets the number of tiles of the lot along its local Z axis

```
public byte SizeZ { get; set; }
```

Property Value

byte

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 2 through 13

Class ModularPiece

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a modular piece of the ModularPiecesSection

```
public sealed class ModularPiece
```

Inheritance

object ← ModularPiece

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The key types below are those of every nonzero key in the lots examined.

Properties

CeilingRailKey

Gets/sets the key of the piece's ceiling rail, a CTProductCeilingRail (0x3F0C529A), or the null key

```
public ResourceKey CeilingRailKey { get; set; }
```

Property Value

ResourceKey

FenceKey

Gets/sets the key of the piece's fence, a CTProductFence (0x0418FE2A) or CTProductWall (0x9151E6BC), or the null key

```
public ResourceKey FenceKey { get; set; }
```

Property Value

ResourceKey

Remarks

The game checks its type against CTProductWall.

FirstWallPatternKey

Gets/sets the first wall pattern key of the piece, a CTProductWallPattern (0xD5F0F921), or the null key, written from version 5

```
public ResourceKey FirstWallPatternKey { get; set; }
```

Property Value

ResourceKey

Remarks

Both wall pattern keys are set or both are null in every lot examined; they differ in 218 of the 278 pieces that have them.

FloorPatternKey

Gets/sets the key of the piece's floor pattern, a CTProductFloorPattern (0xB4F762C9), or the null key

```
public ResourceKey FloorPatternKey { get; set; }
```

Property Value

ResourceKey

Remarks

When it reads a version before 3, the game replaces a key of any other type with a default.

Level

Gets/sets the piece's level, the high byte of Position

```
public sbyte Level { get; set; }
```

Property Value

sbyte

ModularPieceKey

Gets/sets the key of the piece, a CTProductModularPiece (0xA0451CBD)

```
public ResourceKey ModularPieceKey { get; set; }
```

Property Value

ResourceKey

Position

Gets/sets the piece's position, which the game reads as one 32-bit value and also uses as the piece's key

```
public uint Position { get; set; }
```

Property Value

uint

Remarks

The low byte is 0 in the lots examined; the other three are X, Z and Level, from low to high.

SecondWallPatternKey

Gets/sets the second wall pattern key of the piece, a CTProductWallPattern (0xD5F0F921), or the null key, written from version 5

```
public ResourceKey SecondWallPatternKey { get; set; }
```

Property Value

ResourceKey

Unknown1

Gets/sets a key of unknown role

```
public ResourceKey Unknown1 { get; set; }
```

Property Value

ResourceKey

Remarks

The null key in every lot examined.

Unknown2

Gets/sets a value of unknown meaning

```
public byte Unknown2 { get; set; }
```

Property Value

byte

Remarks

0, 2, 4 or 6 in the lots examined; the game passes it to its footprint routine. Hypothesis: the piece's orientation, in eighths of a turn.

Unknown3

Gets/sets a value of unknown meaning, written from version 6

```
public ushort Unknown3 { get; set; }
```

Property Value

ushort

Remarks

0 in 1,254 of the 1,317 version 6 pieces examined, otherwise 8 through 92.

X

Gets/sets the piece's position along the lot's local X axis, in half tiles: the second byte of Position

```
public sbyte X { get; set; }
```

Property Value

sbyte

Remarks

The game multiplies it, as a signed byte, by 0.5.

Z

Gets/sets the piece's position along the lot's local Z axis, in half tiles: the third byte of Position

```
public sbyte Z { get; set; }
```

Property Value

sbyte

Remarks

The game multiplies it, as a signed byte, by 0.5.

Class ModularPiecesSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the ModularPieces section: the modular pieces placed on the lot (🔓)

```
public sealed class ModularPiecesSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← ModularPiecesSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader. Versions 4 through 6 are supported, those in the lots examined, and the game writes version 6. FirstWallPatternKey and SecondWallPatternKey exist from version 5, Unknown3 from version 6.

Constructors

ModularPiecesSection()

Initializes a new instance of the ModularPiecesSection class

```
public ModularPiecesSection()
```

Properties

Pieces

Gets the modular pieces

```
public Collection<ModularPiece> Pieces { get; }
```

Property Value

Collection<ModularPiece>

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

> The version is not 4 through 6

Class ObjectFallbacksSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the ObjectFallbacks section: the game's object fallback data, four sets of object ids (🔓)

```
public sealed class ObjectFallbacksSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← ObjectFallbacksSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game hands this data to its "Object Fallback Manager", the target of a Python function of that name. Version 2 is the only one supported: a byte followed by four categories, each a 32-bit count and that many object ids. Version 1 stores the data as an `ObjectFallbackDataList` message instead; no lot examined has it, so it is rejected.

The game reads exactly four categories whatever Unknown1 says, and loads each id other than 0 into a set, so the order of the ids on disk is the order of the game's hash set and means nothing. The meaning of the categories has not been established. In the lots examined Unknown1 is 4, and each section has two or three ids, all in the second category.

Constructors

ObjectFallbacksSection()

Initializes a new instance of the ObjectFallbacksSection class

```
public ObjectFallbacksSection()
```

Fields

CategoryCount

Gets the number of categories, which is 4

```
public const int CategoryCount = 4
```

Field Value

int

Properties

Categories

Gets the object ids of each of the four categories

```
public IReadOnlyList<Collection<ulong>> Categories { get; }
```

Property Value

IReadOnlyList<Collection<ulong>>

Unknown1

Gets/sets the byte that precedes the categories

```
public byte Unknown1 { get; set; }
```

Property Value

byte

Remarks

The game clamps it to at most 4 and passes it along while inserting ids, but no code in the game's simulation library makes use of it (other parts of the game were not examined). It is 4 in every lot

examined.

Class ObjectTileBounds

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one entry of an ObjectTileBoundsSection: the tiles an object covers

```
public sealed class ObjectTileBounds
```

Inheritance

object ← ObjectTileBounds

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game computes the rectangle from the object's world bounds, rounded out to whole tiles and clipped to the lot. It is stored as MinimumZ, MaximumZ, MinimumX, MaximumX, in that order; each minimum is at or below its maximum in every lot examined.

Constructors

ObjectTileBounds()

Initializes a new instance of the ObjectTileBounds class

```
public ObjectTileBounds()
```

Properties

Level

Gets/sets the signed level of the object

```
public sbyte Level { get; set; }
```

Property Value

sbyte

MaximumX

Gets/sets the highest tile the object covers along the lot's local X axis

```
public byte MaximumX { get; set; }
```

Property Value

byte

Remarks

Present from version 2 on.

MaximumZ

Gets/sets the highest tile the object covers along the lot's local Z axis

```
public byte MaximumZ { get; set; }
```

Property Value

byte

Remarks

Present from version 2 on.

MinimumX

Gets/sets the lowest tile the object covers along the lot's local X axis

```
public byte MinimumX { get; set; }
```

Property Value

byte

Remarks

Present from version 2 on.

MinimumZ

Gets/sets the lowest tile the object covers along the lot's local Z axis

```
public byte MinimumZ { get; set; }
```

Property Value

byte

Remarks

Present from version 2 on.

ObjectId

Gets/sets the id of the object

```
public ulong ObjectId { get; set; }
```

Property Value

ulong

Class ObjectTileBoundsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the ObjectTileBounds section: the tile rectangles of the lot's objects (🔓)

```
public sealed class ObjectTileBoundsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← ObjectTileBoundsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout is a 32-bit entry count and, per entry, the object id, the tile rectangle (from version 2 on) and the level. The game sorts the entries when it writes the section; they are kept here in the order read.

Versions 1 and 2 are supported, and the game writes version 2.

Constructors

ObjectTileBoundsSection()

Initializes a new instance of the ObjectTileBoundsSection class

```
public ObjectTileBoundsSection()
```

Properties

Entries

Gets the entries

```
public Collection<ObjectTileBounds> Entries { get; }
```

Property Value

Collection<ObjectTileBounds>

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

> The version is not 1 or 2

Class PendingPlacementSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the PendingPlacement section: a footprint the game places on the lot after loading (🔓)

```
public sealed class PendingPlacementSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← PendingPlacementSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

When State is 1, the game centers a footprint of Width by Depth tiles, turned by Rotation, on the lot and builds a placement operation with BlueprintInstance; that this places a pending lot template or blueprint is a hypothesis. Every lot examined holds the defaults (state 0).

The layout is Width, Depth, Rotation (from version 3 on), State (a Boolean in version 1), then from version 2 on Unknown1, Unknown2, Unknown3 (from version 4 on) and Unknown4 (from version 5 on). The game reads this section only when Version is 9 or later.

Versions 1 through 5 are supported, and the game writes version 5.

Constructors

PendingPlacementSection()

Initializes a new instance of the PendingPlacementSection class

```
public PendingPlacementSection()
```

Properties

Depth

Gets/sets the second dimension of the footprint, in tiles

```
public byte Depth { get; set; }
```

Property Value

byte

Rotation

Gets/sets the rotation of the footprint relative to the lot, in quarter turns

```
public byte Rotation { get; set; }
```

Property Value

byte

Remarks

Present from version 3 on. The game uses it modulo 4.

State

Gets/sets the state of the placement; the game places the footprint only when it is 1

```
public byte State { get; set; }
```

Property Value

byte

Remarks

In version 1 it is stored as a Boolean, so it must be 0 or 1 to be written.

Unknown1

Gets/sets the first unknown value

```
public bool Unknown1 { get; set; }
```

Property Value

bool

Remarks

Present from version 2 on. It is true in every lot examined.

Unknown2

Gets/sets the second unknown value

```
public ulong Unknown2 { get; set; }
```

Property Value

ulong

Remarks

Present from version 2 on. It is 0 in every lot examined.

Unknown3

Gets/sets the third unknown value

```
public bool Unknown3 { get; set; }
```

Property Value

bool

Remarks

Present from version 4 on. It is false in every lot examined.

Unknown4

Gets/sets the fourth unknown value

```
public bool Unknown4 { get; set; }
```

Property Value

bool

Remarks

Present from version 5 on. It is false in every lot examined.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 1 through 5

Width

Gets/sets the first dimension of the footprint, in tiles

```
public byte Width { get; set; }
```

Property Value

byte

Remarks

Which lot axis each dimension runs along depends on the rotations of the footprint and the lot.

Class Plex

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a plex (a unit of a multi-unit lot) of the PlexesSection, with the blocks that make up its outline

```
public sealed class Plex
```

Inheritance

object ← Plex

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader and writer.

Layout, with gates on the section's version: Id (ushort), the offset within the section at which the plex ends (uint, from version 2 on), IsVisible (uint), a uint block count, the Blocks, Unknown1 and Unknown2 (after 4), the name (version 6: a uint byte count and ASCII text), then after 5 TileCount, Unknown3, Unknown4 and Unknown5. The game writes the end offset as a placeholder and patches it, so that its reader can skip the plex; the offset is not a property, it is checked on reading and computed on writing.

Properties

Blocks

Gets the blocks of the plex

```
public Collection<WallBlock> Blocks { get; }
```

Property Value

Collection<WallBlock>

Remarks

In the lots examined 1,182 of the 1,363 blocks of plexes have a Id that is also the id of a block of the bundle's WallBlocksSection; version 4 sections hold block versions 32, 33 and 35, later ones version 36.

Id

Gets/sets the id of the plex

```
public ushort Id { get; set; }
```

Property Value

ushort

Remarks

The game keys its plexes by this id; 300 is the shared plex.

IsVisible

Gets/sets whether the plex is visible

```
public bool IsVisible { get; set; }
```

Property Value

bool

Remarks

The game keeps the low byte at offset 0x44 of the plex, which its VisibleCallback compares with 1.

Name

Gets/sets the name version 6 stores

```
public string Name { get; set; }
```

Property Value

string

Remarks

The game reads it and discards it. Observed values are "Shared Unit" and "Unit 1". It is stored as a byte count and one byte per character, so it must be ASCII: reading a name with other bytes fails, and so does encoding a name with other characters.

TileCount

Gets/sets the number of tiles of the plex

```
public uint TileCount { get; set; }
```

Property Value

uint

Remarks

The game returns it from its get_plex_tile_count Python function. Stored from version 6 on.

Unknown1

Gets/sets the first of two ulong values the game keeps at offsets 0x30 and 0x38 of the plex

```
public ulong Unknown1 { get; set; }
```

Property Value

ulong

Remarks

Hypothesis: the id of the owning household; where it is 0 the game falls back to the lot owner. The game zeroes both values when loading with one of its load flags. 0 in every lot examined. Stored from version 5 on.

Unknown2

Gets/sets the second of two ulong values the game keeps at offsets 0x30 and 0x38 of the plex

```
public ulong Unknown2 { get; set; }
```

Property Value

ulong

Remarks

0 in every lot examined. Stored from version 5 on.

Unknown3

Gets/sets a byte the game keeps at offset 0x45 of the plex

```
public byte Unknown3 { get; set; }
```

Property Value

byte

Remarks

The game treats it as bit flags. 0, 1 or 10 in the lots examined. Stored from version 6 on.

Unknown4

Gets/sets a byte the game keeps at offset 0x46 of the plex

```
public byte Unknown4 { get; set; }
```

Property Value

byte

Remarks

The game copies it from the plex's zone object. 0 in every lot examined. Stored from version 6 on.

Unknown5

Gets/sets a byte the game keeps at offset 0x47 of the plex

```
public byte Unknown5 { get; set; }
```

Property Value

byte

Remarks

The game copies it from the plex's zone object. 0 in every lot examined. Stored from version 6 on.

Class PlexGroup

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a group of the PlexGroupsSection

```
public sealed class PlexGroup
```

Inheritance

object ← PlexGroup

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader and writer.

Layout: a ushort group version (the game writes 1 and fails to load the lot when it is 0; 1 in every lot examined, the only one supported), Id, Unknown1, a ushort count, then the Values.

Properties

Id

Gets/sets the id of the group, by which GroupId refers to it

```
public uint Id { get; set; }
```

Property Value

uint

Remarks

In every lot examined the groups of a section have the ids 0, 1 and 2, in order.

Unknown1

Gets/sets a byte of the group

```
public byte Unknown1 { get; set; }
```

Property Value

byte

Remarks

In every lot examined the three groups of a section have a permutation of 0, 1 and 2.

Values

Gets the values of the group

```
public Collection<ushort> Values { get; }
```

Property Value

Collection<ushort>

Remarks

Hypothesis: block ids. They are strictly ascending, 0 through 307, in the lots examined. 469 of the 736 groups examined are empty; of the 3,522 values of the other 267, 3,271 are ids of blocks of the bundle's WallBlocksSection, and in 21 of those groups every value is.

Class PlexGroupMap

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of the map from which the game looks up groups of the PlexGroupsSection

```
public sealed class PlexGroupMap
```

Inheritance

object ← PlexGroupMap

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game looks a group id up by Unknown1 and a MapKey. Layout: Unknown1, a ushort count, then the Entries.

Properties

Entries

Gets the entries

```
public Collection<PlexGroupMapEntry> Entries { get; }
```

Property Value

Collection<PlexGroupMapEntry>

Remarks

3 entries with the keys 0, 1 and 2, or 1 entry, in the lots examined.

Unknown1

Gets/sets the outer key of the map entry

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Remarks

Hypothesis: a plex id. 0 in every lot examined.

Class PlexGroupMapEntry

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of a PlexGroupMap

```
public sealed class PlexGroupMapEntry
```

Inheritance

object ← PlexGroupMapEntry

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

GroupId

Gets/sets the Id of the group

```
public uint GroupId { get; set; }
```

Property Value

uint

Remarks

It is the id of a group of the bundle's PlexGroupsSection in every lot examined.

MapKey

Gets/sets the inner key of the map entry

```
public sbyte MapKey { get; set; }
```

Property Value

sbyte

Remarks

Hypothesis: an index of exterior variants, which an operation of the game swaps.

Class PlexGroupsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the PlexGroups section: groups of values the PlexesSection refers to by id (🔓)

```
public sealed class PlexGroupsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← PlexGroupsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader and writer; the game reads the section only when the LotInfo Section is version 20 or later, and after loading forwards the groups to a global service. What the groups mean is unknown; that their values are block ids is a hypothesis (see Values).

Version 1 is the only one supported; the game writes it, and the lots examined hold only it. The game reads nothing more from a section of version 0, and the layout of every other version as that of version 1.

Layout: the version (uint), a ushort count, then the Groups.

Constructors

PlexGroupsSection()

Initializes a new instance of the PlexGroupsSection class

```
public PlexGroupsSection()
```

Properties

Groups

Gets the groups

```
public Collection<PlexGroup> Groups { get; }
```

Property Value

Collection <PlexGroup>

Remarks

0, 1 or 3 in the lots examined.

Class PlexesSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Plexes section: the lot's plexes (units), with the blocks of their outlines (🔓)

```
public sealed class PlexesSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← PlexesSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader and writer; the game reads the section only when the LotInfo Section is version 14 or later. Versions 1 through 9 are supported, and the game writes version 9.

Layout: Version, Unknown1 (after 2), a uint plex count, the Plexes, then after 7 a uint count of Unknown2, and only if it is not 0: Unknown2, Unknown3, a uint count and Unknown4; then after 8 a ushort count and the GroupMaps.

Constructors

PlexesSection()

Initializes a new instance of the PlexesSection class

```
public PlexesSection()
```

Properties

GroupMaps

Gets the maps from which the game looks up groups of the PlexGroupsSection

```
public Collection<PlexGroupMap> GroupMaps { get; }
```

Property Value

Collection<PlexGroupMap>

Remarks

0 or 1 entries in the lots examined. Stored from version 9 on.

Plexes

Gets the plexes

```
public Collection<Plex> Plexes { get; }
```

Property Value

Collection<Plex>

Remarks

0 through 5 in the lots examined.

Unknown1

Gets/sets a Boolean the game keeps at offset 0x115 of its plex manager

```
public bool Unknown1 { get; set; }
```

Property Value

bool

Remarks

While it is set, the game runs a routine of its plex manager and then clears it; the game sets it for sections before version 4. Hypothesis: the plex outlines need regenerating. 0 in every lot examined. Stored from version 3 on.

Unknown2

Gets the first of the extra lists of blocks of versions 8 and later

```
public Collection<WallBlock> Unknown2 { get; }
```

Property Value

Collection<WallBlock>

Remarks

The game keeps them at offset 200 of its plex manager. Empty in every lot examined.

Unknown3

Gets/sets the block stored after Unknown2

```
public WallBlock? Unknown3 { get; set; }
```

Property Value

WallBlock

Remarks

The game keeps it at offset 0x30 of its plex manager. It is stored only when Unknown2 is not empty, and must then be set.

Unknown4

Gets the second of the extra lists of blocks of versions 8 and later

```
public Collection<WallBlock> Unknown4 { get; }
```

Property Value

Collection<WallBlock>

Remarks

The game keeps them at offset 0xE0 of its plex manager. It is stored only when Unknown2 is not empty.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

> The version is not 1 through 9

Class Pond

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a pond of the PondsSection

```
public sealed class Pond
```

Inheritance

object ← Pond

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader.

Properties

Cells

Gets the tile quarters the pond covers

```
public Collection<PondCell> Cells { get; }
```

Property Value

Collection<PondCell>

FloorPatternKey

Gets/sets the key of the pond's floor pattern, a CTProductFloorPattern (0xB4F762C9)

```
public ResourceKey FloorPatternKey { get; set; }
```

Property Value

ResourceKey

Remarks

The PondBottoms grid holds this key's id on the pond's quarters in the lots examined.

Id

Gets/sets the pond's id

```
public ushort Id { get; set; }
```

Property Value

ushort

Remarks

The game keys its pond map and its cell-to-pond map by it; 1, 2, 3 or 11 in the lots examined.

Unknown2

Gets a list of values of unknown meaning, written from version 3

```
public Collection<ushort> Unknown2 { get; }
```

Property Value

Collection<ushort>

Remarks

The game reads it into a set; it is empty in every lot examined.

WaterLevel

Gets/sets the pond's water level, in steps of 0.25 m

```
public int WaterLevel { get; set; }
```

Property Value

int

Remarks

It equals GetWaterLevel(int, int, FloorQuarter) on every quarter of the pond, which the game multiplies by 0.25. Creating a pond in game set it to −1, and raising the pond's water level one step changed it from −3 to −2 together with the water level of every quarter of the pond in the TerrainHeightsSection, while the pond spread to more cells (in-game tests). It is −1, −2, −3 or 0 in the lots examined.

Class PondCell

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a tile of a Pond and the quarters of it the pond covers

```
public sealed class PondCell
```

Inheritance

object ← PondCell

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game keys a cell by (X, Z, Level, QuarterMask), so one tile can appear once per quarter.

Properties

Level

Gets/sets the tile's level

```
public sbyte Level { get; set; }
```

Property Value

sbyte

Remarks

0 in the lots examined.

QuarterMask

Gets/sets the quarters of the tile the pond covers, bit `n` standing for the FloorQuarter with value `n`

```
public byte QuarterMask { get; set; }
```

Property Value

byte

Remarks

1, 2, 4, 8 or 15 in the lots examined.

Unknown1

Gets/sets a value of unknown meaning, written from version 3

```
public byte Unknown1 { get; set; }
```

Property Value

byte

Remarks

The game sets it to 3 when reading an older version; it is 3 in every lot examined and was 3 in every cell of the ponds of the in-game tests.

X

Gets/sets the tile's index along the lot's local X axis

```
public byte X { get; set; }
```

Property Value

byte

Z

Gets/sets the tile's index along the lot's local Z axis

```
public byte Z { get; set; }
```

Property Value

byte

Class PondsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Ponds section: the lot's ponds and the tile quarters each one covers (🔓)

```
public sealed class PondsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← PondsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Versions 0 through 3 are supported, as the game's reader accepts, and the game writes version 3, the only one in the lots examined. Versions 0 and 1 hold no ponds; Unknown1 and Unknown2 exist from version 3.

Constructors

PondsSection()

Initializes a new instance of the PondsSection class

```
public PondsSection()
```

Properties

Ponds

Gets the ponds

```
public Collection<Pond> Ponds { get; }
```

Property Value

Collection<Pond>

Remarks

The section holds ponds from version 2; in versions 0 and 1 this collection must be empty.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

> The version is not 0 through 3

Class ProductTable

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a table of catalog product keys and the ids by which the rest of a section refers to them

```
public sealed class ProductTable
```

Inheritance

object ← ProductTable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader and writer. When the game adds a key, it gives a new key the lowest id from 1 up that is not in use, and adds the added amount to the ReferenceCount of a key that is already present. Id 0 stands for DefaultKey: the table returns 0 for that key and never adds it.

Layout: DefaultKey (instance, type, group), a uint count, then the Entries.

Properties

DefaultKey

Gets/sets the key that id 0 stands for

```
public ResourceKey DefaultKey { get; set; }
```

Property Value

ResourceKey

Remarks

It is the null key in every lot examined.

Entries

Gets the entries, in the order the game wrote them

```
public Collection<ProductTableEntry> Entries { get; }
```

Property Value

Collection <ProductTableEntry>

Class ProductTableEntry

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of a ProductTable: a catalog product key and its id

```
public sealed class ProductTableEntry
```

Inheritance

object ← ProductTableEntry

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Key (instance, type, group), Id, ReferenceCount.

Properties

Id

Gets/sets the id by which the rest of the section refers to Key

```
public uint Id { get; set; }
```

Property Value

uint

Remarks

Ids are unique within a table and never 0 in the lots examined; they run 1 through the entry count in 38,097 of the 68,278 non-empty tables examined (56%).

Key

Gets/sets the key of the catalog product

```
public ResourceKey Key { get; set; }
```

Property Value

ResourceKey

ReferenceCount

Gets/sets the reference count the game keeps for the key

```
public uint ReferenceCount { get; set; }
```

Property Value

uint

Remarks

The game adds to it each time the key is added to the table again. It does not always equal the number of references in the section: in 63,616 of the 63,742 non-empty block product tables examined every reference count is 1.

Class ResourceManifestSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the ResourceManifest section: the keys of the catalog resources the other sections reference (🔓)

```
public sealed class ResourceManifestSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← ResourceManifestSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game builds the manifest while saving: it records every key it writes in the sections that precede the manifest in its save order, skipping repeats, so the manifest starts with the lot's style key and the null key. GetManifestKeys() does the same with the current sections. The game writes the manifest, but no code in the game's simulation library reads it back (other parts of the game were not examined). The section has no version.

Constructors

ResourceManifestSection()

Initializes a new instance of the ResourceManifestSection class

```
public ResourceManifestSection()
```

Properties

Keys

Gets the keys

```
public Collection<ResourceKey> Keys { get; }
```

Property Value

Collection <ResourceKey>

Class RevisionSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Revision section: the lot's revision counter (🔓)

```
public sealed class RevisionSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← RevisionSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game keeps the counter on the lot, starts it at 1 and increments it whenever the lot is modified, for example when the height of a level changes. It writes the section only when a save flag is set.

The section has no version. In the lots examined the value fits in 32 bits, from 1 to 587,252.

Constructors

RevisionSection()

Initializes a new instance of the RevisionSection class

```
public RevisionSection()
```

Properties

Value

Gets/sets the revision counter

```
public ulong Value { get; set; }
```

Property Value

ulong

Class Roof

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one roof of a RoofsSection

```
public sealed class Roof
```

Inheritance

object ← Roof

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

From version 10 of the section on, a roof holds an InnerVersion, the bounding box of its footprint, the four corners of its footprint, Direction, Pitch, Curvature, four more floats (inner version 3 and later), Base Elevation, four per-side bytes (one byte copied to all four before inner version 2) and Flags. Before version 10, a roof holds only the bounding box (as two three-byte points), Direction, one per-side byte copied to all four, Curvature, Pitch and, in version 9, BaseElevation; the game derives the corners and computes the base elevation of older roofs while loading.

The game replaces each key whose catalog resource is missing or invalid with the default of its type while loading.

Constructors

Roof()

Initializes a new instance of the Roof class

```
public Roof()
```

Properties

BaseElevation

Gets/sets the elevation of the roof's base

```
public float BaseElevation { get; set; }
```

Property Value

float

Remarks

Present in version 9 and from version 10 on. When it is outside the valid range, the game recomputes it while loading as the elevation of the roof's level plus the lot's LevelElevationOffset (levels 0 and up) or NegativeLevelElevationOffset (levels below 0). Raising the walls below a roof from 3 m to 5 m changed it from 3 to 5 (in-game test).

BoundsMaximum

Gets/sets the maximum corner of the bounding box of the roof's footprint

```
public RoofPoint BoundsMaximum { get; set; }
```

Property Value

RoofPoint

Remarks

It is at or above BoundsMinimum on both axes in every lot examined.

BoundsMinimum

Gets/sets the minimum corner of the bounding box of the roof's footprint

```
public RoofPoint BoundsMinimum { get; set; }
```

Property Value

RoofPoint

Corner1

Gets/sets the first corner of the roof's footprint

```
public RoofPoint Corner1 { get; set; }
```

Property Value

RoofPoint

Remarks

Present from version 10 on. The game derives the four corners from the bounding box; with bit 0 of Flags set, they are inset by half the extent, giving a footprint turned 45°.

Corner2

Gets/sets the second corner of the roof's footprint

```
public RoofPoint Corner2 { get; set; }
```

Property Value

RoofPoint

Remarks

Present from version 10 on.

Corner3

Gets/sets the third corner of the roof's footprint

```
public RoofPoint Corner3 { get; set; }
```

Property Value

RoofPoint

Remarks

Present from version 10 on.

Corner4

Gets/sets the fourth corner of the roof's footprint

```
public RoofPoint Corner4 { get; set; }
```

Property Value

RoofPoint

Remarks

Present from version 10 on.

Curvature

Gets/sets the curvature of the roof

```
public float Curvature { get; set; }
```

Property Value

float

Remarks

In-game tests show that curving a roof changes only this value (from 0 to 0.5604 in one test). It is signed, −1 through 1, and defaults to 0.

Direction

Gets/sets the direction of the roof, a value of the game's eight-way direction enumeration

```
public byte Direction { get; set; }
```

Property Value

byte

Remarks

The game rotates it in steps of the eight-way enumeration (modulo 8); it is 0, 2, 4 or 6 in every lot examined. Turning a gable roof 90° in game changed it from 0 to 6, two steps of the enumeration (in-game test). That it is the direction of the roof's ridge is a hypothesis.

FirstTrimKey

Gets/sets the key of the first trim, a `CTProductRoofTrim` (0xB0311D0F) or the null key

```
public ResourceKey FirstTrimKey { get; set; }
```

Property Value

ResourceKey

Remarks

Changing the roof trim in game left it the null key and changed only SecondTrimKey (in-game test).

Flags

Gets/sets the flags of the roof

```
public uint Flags { get; set; }
```

Property Value

uint

Remarks

Present from version 10 on. Bit 0 insets the footprint's corners into a diamond turned 45°; the other bits are unknown. Placing the catalog's diagonal gable roof set bit 0 and inset the corners, as the game's code describes (in-game test). It is 0, 1 or 2 in every lot examined.

Id

Gets/sets the id of the roof

```
public uint Id { get; set; }
```

Property Value

uint

InnerVersion

Gets/sets the version of the roof's own layout

```
public uint InnerVersion { get; set; }
```

Property Value

uint

Remarks

Present from version 10 of the section on; for older sections it keeps its default, 3, and encoding throws if it is changed. The game writes 3 and its reader reads any value, laying out the roof as for 3 when the value is 3 or more; this library supports 0 through 3. In the lots examined the roofs of version 11 sections hold 2 and those of version 13 sections 3; no version 10 or 12 section examined holds roofs. Inner version 3 holds Unknown1 through Unknown4, which encoding a lower inner version requires to keep their defaults; before inner version 2, a single per-side byte is stored and copied to Unknown5 through Unknown8.

Exceptions

ArgumentOutOfRangeException

The inner version is not 0 through 3

PatternKey

Gets/sets the key of the pattern, a `CTProductRoofPattern` (0xF1EDBD86)

```
public ResourceKey PatternKey { get; set; }
```

Property Value

ResourceKey

Remarks

Present from version 6 on. Changing the shingles in game changed only this key (in-game test).

Pitch

Gets/sets the pitch of the roof

```
public float Pitch { get; set; }
```

Property Value

float

Remarks

The game compares it with the `MaxRoofPitch` build-buy tuning (reporting `MaxRoofSlopeExceeded`); one pitch step in the game adds 0.05. It defaults to 0.5 and is 0.05 through 1.75 in the lots examined.

RoofKey

Gets/sets the key of the roof, a `CTProductRoof` (0x91EDBD3E), which decides its shape

```
public ResourceKey RoofKey { get; set; }
```

Property Value

ResourceKey

Remarks

In-game tests show that the shape is this product alone: replacing one shape with another changed only this key and the per-side bytes Unknown5 through Unknown8. The instances seen are 0x144D (gable), 0x1B03 (half gable), 0x144F (hip), 0x29DC (half hip), 0x282AD (octagonal), 0x282B8 (cone) and 0x7DD4 (the catalog's diagonal gable).

SecondTrimKey

Gets/sets the key of the second trim, a `CTProductRoofTrim` (0xB0311D0F)

```
public ResourceKey SecondTrimKey { get; set; }
```

Property Value

ResourceKey

Remarks

Changing the roof trim in game changed only this key (in-game test).

Unknown1

Gets/sets the first unknown float

```
public float Unknown1 { get; set; }
```

Property Value

float

Remarks

Present from inner version 3 on; the game's default is 0.66. The four unknown floats look like the parameters of a curve profile (hypothesis). All four kept their defaults (0.66, 0.33, −1 and 1) through

every in-game test.

Unknown2

Gets/sets the second unknown float

```
public float Unknown2 { get; set; }
```

Property Value

float

Remarks

Present from inner version 3 on; the game's default is 0.33.

Unknown3

Gets/sets the third unknown float

```
public float Unknown3 { get; set; }
```

Property Value

float

Remarks

Present from inner version 3 on; the game's default is −1.

Unknown4

Gets/sets the fourth unknown float

```
public float Unknown4 { get; set; }
```

Property Value

float

Remarks

Present from inner version 3 on; the game's default is 1.

Unknown5

Gets/sets the first per-side byte

```
public byte Unknown5 { get; set; }
```

Property Value

byte

Remarks

The four per-side bytes are 0 through 12 in the lots examined, and 0 through 3 in roofs of sections before version 13. In in-game tests they were 1, 1, 1, 1 for the gable, half gable and diagonal gable roofs and 0, 0, 0, 0 for the hip, half hip, octagonal and cone roofs, and replacing one shape with another changed only these bytes and RoofKey; that they choose gable or eave per side is a hypothesis, and their meaning is not proven. Before version 10 and before inner version 2, a single byte is stored, so the four must be equal to be written.

Unknown6

Gets/sets the second per-side byte

```
public byte Unknown6 { get; set; }
```

Property Value

byte

Unknown7

Gets/sets the third per-side byte

```
public byte Unknown7 { get; set; }
```

Property Value

byte

Unknown8

Gets/sets the fourth per-side byte

```
public byte Unknown8 { get; set; }
```

Property Value

byte

WallKey

Gets/sets the wall key, a `CTProductWallPattern` (0xD5F0F921) or the null key

```
public ResourceKey WallKey { get; set; }
```

Property Value

ResourceKey

Remarks

Present from version 11 on.

Struct RoofPoint

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a lot-local tile position of a Roof's footprint

```
public readonly record struct RoofPoint : IEquatable<RoofPoint>
```

Implements
IEquatable<RoofPoint>

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Remarks

From version 10 of RoofsSection on, each point is stored as a 32-bit value: X in the low byte, then Z, then Level, then Reserved in the high byte. Before version 10, a point is stored as its three low bytes.

Properties

Level

Gets/sets the signed lot level

```
public sbyte Level { get; init; }
```

Property Value

sbyte

Reserved

Gets/sets the high byte of the stored value

```
public byte Reserved { get; init; }
```

Property Value

byte

Remarks

The game's reader discards it and its writer writes 0; it is 0 in every lot examined.

X

Gets/sets the tile position along the lot's local X axis

```
public byte X { get; init; }
```

Property Value

byte

Z

Gets/sets the tile position along the lot's local Z axis

```
public byte Z { get; init; }
```

Property Value

byte

Class RoofsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Roofs section: the lot's roofs (🔓)

```
public sealed class RoofsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← RoofsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The header holds LegacyKey (before version 8), ReservedKey, Unknown1, Unknown2, Unknown3 and the roof count (a 32-bit value); the roofs follow. The layout of each roof depends on the version, as Roof describes.

Versions 5 through 13 are supported, and the game writes version 13.

In-game tests show that a roof's shape is decided by its RoofKey alone, with the per-side bytes following the shape; Roof lists the observations.

Constructors

RoofsSection()

Initializes a new instance of the RoofsSection class

```
public RoofsSection()
```

Properties

LegacyKey

Gets/sets the legacy key of the header

```
public ResourceKey LegacyKey { get; set; }
```

Property Value

ResourceKey

Remarks

Present before version 8. The game's reader discards it.

ReservedKey

Gets/sets the reserved key of the header

```
public ResourceKey ReservedKey { get; set; }
```

Property Value

ResourceKey

Remarks

The game's writer writes the null key (recording it for the ResourceManifestSection) and its reader discards it; it is the null key in every lot examined.

Roofs

Gets the roofs

```
public Collection<Roof> Roofs { get; }
```

Property Value

Collection <Roof>

Unknown1

Gets/sets the first unknown value of the header

```
public float Unknown1 { get; set; }
```

Property Value

float

Remarks

The game keeps it in its roof manager; it is 0.5 in every lot examined.

Unknown2

Gets/sets the second unknown value of the header

```
public uint Unknown2 { get; set; }
```

Property Value

uint

Remarks

The game's writer writes 0 and its reader discards it. It is 0 in the lots examined except in three version 6 and 8 records (45, 45 and 5), where it may count roof ids (hypothesis: 45 is the highest roof id there).

Unknown3

Gets/sets the third unknown value of the header

```
public bool Unknown3 { get; set; }
```

Property Value

bool

Remarks

The game's writer writes true and its reader discards it; it is true in every lot examined.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

> The version is not 5 through 13

Class Stair

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one stair or ladder of a StairsSection

```
public sealed class Stair
```

Inheritance

object ← Stair

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Each stair is also a lot object: ObjectId is the id of its object in LotObjectsSection.

Before version 7, a stair holds a second position and its bottom level (LegacyX, LegacyZ, BottomLevel) ahead of AnchorX, AnchorZ and TopLevel, and no stages: the game derives one stage from the distance between the two positions and computes TopElevation while loading. Versions 7 through 9 hold exactly one stage and versions 10 and later a 16-bit stage count. Before version 8, Flags is stored as separate Booleans (four, or six from version 6 on).

Constructors

Stair()

Initializes a new instance of the Stair class

```
public Stair()
```

Properties

AnchorX

Gets/sets the position of the stair's centerline anchor along the lot's local X axis, in tiles

```
public float AnchorX { get; set; }
```

Property Value

float

Remarks

Its fraction is 0 or 0.5 in the lots examined. Which end of the stair it marks has not been established.

AnchorZ

Gets/sets the position of the stair's centerline anchor along the lot's local Z axis, in tiles

```
public float AnchorZ { get; set; }
```

Property Value

float

Remarks

Widening a stair in the game by one lane moved it by 0.5.

BottomLevel

Gets/sets the signed level at the bottom of the stair

```
public sbyte BottomLevel { get; set; }
```

Property Value

sbyte

Remarks

Before version 7 it is stored as a 32-bit value next to LegacyX and LegacyZ.

FirstStep

Gets/sets the first step of the stair's step range

```
public ushort FirstStep { get; set; }
```

Property Value

ushort

Remarks

Present from version 14 on. The game clamps the stages to the range FirstStep through LastStep. It is 0 through 5 in the lots examined.

Flags

Gets/sets the flags of the stair

```
public ushort Flags { get; set; }
```

Property Value

ushort

Remarks

Before version 8, bits 0 through 3 (and bits 4 and 5 from version 6 on) are stored as Booleans, and the other bits cannot be stored. The game recomputes bits 4 and 5 while loading when Unknown1 is set, and sets bits 11 and 12 when bit 0 is set and LastStep is 65,535; the meaning of the bits is otherwise unknown. Changing the railing in game set bits 0x0001 and 0x1800 together with RailingKey (in-game test).

Id

Gets/sets the id of the stair, unique among the stairs and ladders of the lot

```
public ushort Id { get; set; }
```

Property Value

ushort

LaneCount

Gets/sets the number of lanes of the stair: its width in tiles

```
public byte LaneCount { get; set; }
```

Property Value

byte

LastStep

Gets/sets the last step of the stair's step range

```
public ushort LastStep { get; set; }
```

Property Value

ushort

Remarks

Present from version 14 on. When it is 65,535, the game computes it while loading as the total step count minus 1 (or the total with bit 3 of Flags set). It was the total step count minus 1 in every in-game test (11 for 12 steps, 19 for 20).

LegacyX

Gets/sets a second position of the stair along the lot's local X axis, in tiles

```
public float LegacyX { get; set; }
```

Property Value

float

Remarks

Present before version 7. The game derives the step count of the stair's single stage from its distance to AnchorX and AnchorZ; that it marks the bottom end is a hypothesis.

LegacyZ

Gets/sets a second position of the stair along the lot's local Z axis, in tiles

```
public float LegacyZ { get; set; }
```

Property Value

float

Remarks

Present before version 7.

ObjectId

Gets/sets the id of the stair's lot object

```
public ulong ObjectId { get; set; }
```

Property Value

ulong

RailingKey

Gets/sets the key of the railing, a `CTProductRailing` (0x1C1CF1F7) or the null key

```
public ResourceKey RailingKey { get; set; }
```

Property Value

ResourceKey

Remarks

It is the null key for every ladder examined. Changing the railing in game set this key and bits 0x0001 and 0x1800 of Flags (in-game test).

SecondRailingKey

Gets/sets the key of the second railing, a `CTProductRailing` (0x1C1CF1F7) or the null key

```
public ResourceKey SecondRailingKey { get; set; }
```

Property Value

ResourceKey

Remarks

Present from version 12 on.

Stages

Gets the stages, ordered from the top

```
public Collection<StairStage> Stages { get; }
```

Property Value

Collection <StairStage>

Remarks

Versions 7 through 9 hold exactly one stage; versions before 7 hold none.

StairsKey

Gets/sets the key of the stair, a `CTProductStair` (0x9A20CD1C) for a stair or a `CTProductLadder` (0xCC09004D) for a ladder

```
public ResourceKey StairsKey { get; set; }
```

Property Value

ResourceKey

TopElevation

Gets/sets the elevation at the top of the stair

```
public float TopElevation { get; set; }
```

Property Value

float

Remarks

Present from version 7 on. The game places each stage's start at this elevation minus the steps above it times the step height; this value minus 0.25 per step is the bottom elevation in most lots examined (hypothesis: the step height is 0.25). Raising the walls of the stair's level from 3 m to 5 m changed it from 3 to 5 and the total step count from 12 to 20 (in-game test).

TopLevel

Gets/sets the signed level at the top of the stair

```
public int TopLevel { get; set; }
```

Property Value

int

Unknown1

Gets/sets the first unknown value

```
public bool Unknown1 { get; set; }
```

Property Value

bool

Remarks

It is true in every stair and ladder examined except four stairs of version 5 and four of version 15.

Unknown2

Gets/sets the second unknown value

```
public float Unknown2 { get; set; }
```

Property Value

float

Remarks

Present in versions 5 and 6. The game converts it into the stair's bottom elevation while loading (hypothesis: a legacy bottom elevation).

Unknown3

Gets/sets the third unknown value

```
public byte Unknown3 { get; set; }
```

Property Value

byte

Remarks

Present in versions 13 and 14. The game's reader discards it.

Unknown4

Gets/sets the fourth unknown value

```
public float Unknown4 { get; set; }
```

Property Value

float

Remarks

Present from version 13 on. It is 0, 0.5 or 1 in the lots examined.

Unknown5

Gets/sets the fifth unknown value

```
public float Unknown5 { get; set; }
```

Property Value

float

Remarks

Present from version 13 on. It is 0, 0.5 or 1 in the lots examined.

Unknown6

Gets/sets the sixth unknown value

```
public float Unknown6 { get; set; }
```

Property Value

float

Remarks

Present from version 13 on. It is 0 in every lot examined.

Unknown7

Gets/sets the seventh unknown value

```
public float Unknown7 { get; set; }
```

Property Value

float

Remarks

Present from version 13 on. It is 0 in every lot examined.

Unknown8

Gets/sets the eighth unknown value

```
public byte Unknown8 { get; set; }
```

Property Value

byte

Remarks

Present from version 15 on. It is 0 or 4 in the lots examined.

WallKey

Gets/sets the wall key, a `CTProductWallPattern` (0xD5F0F921) or the null key

```
public ResourceKey WallKey { get; set; }
```

Property Value

ResourceKey

Remarks

It is the null key for every ladder examined.

Class StairStage

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one stage of a Stair: a straight run of steps

```
public sealed class StairStage
```

Inheritance

object ← StairStage

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

A stage is stored as a 16-bit value holding StepCount in its low 10 bits and a byte (Unknown1). From version 13 of StairsSection on, each stage also has Unknown2 and Unknown3, and from version 15 on Unknown4; these follow the stair's other fields, one set per stage in stage order.

The game keeps only the low 10 bits of StoredStepCount, Unknown2 and Unknown3, the low 6 bits of Unknown1 and the low 4 bits of Unknown4, and writes only those; the stored values are kept whole here so that every record round-trips.

In-game tests: a straight flight of 12 steps had one stage with Unknown2 5 and Unknown3 7; turning its upper part 90° split it into a stage of 8 steps (Unknown2 8, Unknown3 0) and one of 4 (Unknown2 0, Unknown3 4), both with Unknown1 2; after taller walls and a landing, the stages had 12 steps (Unknown1 2) and 8 (Unknown1 4). These are observations; the meanings of the values are not proven.

Constructors

StairStage()

Initializes a new instance of the StairStage class

```
public StairStage()
```

Properties

StepCount

Gets/sets the number of steps of the stage, the low 10 bits of StoredStepCount

```
public ushort StepCount { get; set; }
```

Property Value

ushort

Exceptions

ArgumentOutOfRangeException

The value is greater than 1,023

StoredStepCount

Gets/sets the stored 16-bit value whose low 10 bits are StepCount

```
public ushort StoredStepCount { get; set; }
```

Property Value

ushort

Remarks

The high 6 bits are 0 in every lot examined.

Unknown1

Gets/sets the unknown byte of the stage

```
public byte Unknown1 { get; set; }
```

Property Value

byte

Remarks

The game reads its low 6 bits; it is 0, 2, 4 or 6 in the lots examined, except 12 in 90 stair stages. That it is the stage's direction in the game's eight-way direction enumeration is a hypothesis; it did not change when a flight was turned 90° in game (in-game test).

Unknown2

Gets/sets the first unknown 16-bit value of the stage

```
public ushort Unknown2 { get; set; }
```

Property Value

ushort

Remarks

Present from version 13 on. The game reads its low 10 bits; it looks like a step index (hypothesis).

Unknown3

Gets/sets the second unknown 16-bit value of the stage

```
public ushort Unknown3 { get; set; }
```

Property Value

ushort

Remarks

Present from version 13 on. The game reads its low 10 bits; it looks like a step index (hypothesis).

Unknown4

Gets/sets the unknown byte that follows Unknown3

```
public byte Unknown4 { get; set; }
```

Property Value

byte

Remarks

Present from version 15 on. The game reads its low 4 bits.

Class StairsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Stairs section: the lot's stairs and ladders (🔓)

```
public sealed class StairsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← StairsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The header holds Unknown1 (from version 9 on) and the stair count (8 bits before version 9, 16 bits from then on); the stairs follow, and from version 11 on a 16-bit ladder count and the ladders. The layout of each stair or ladder depends on the version, as Stair describes.

Versions 4 through 15 are supported, and the game writes version 15.

Constructors

StairsSection()

Initializes a new instance of the StairsSection class

```
public StairsSection()
```

Properties

Ladders

Gets the ladders

```
public Collection<Stair> Ladders { get; }
```

Property Value

Collection<Stair>

Remarks

Present from version 11 on.

Stairs

Gets the stairs

```
public Collection<Stair> Stairs { get; }
```

Property Value

Collection<Stair>

Unknown1

Gets/sets the unknown value of the header

```
public bool Unknown1 { get; set; }
```

Property Value

bool

Remarks

Present from version 9 on; the game sets it for older versions. When it is set, the game recomputes bits 4 and 5 of each stair's Flags while loading.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 4 through 15

Class TerrainHeightsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the TerrainHeights section: the lot's terrain height grid, a second vertex grid of unknown role and a water level per tile quarter (🔓)

```
public sealed class TerrainHeightsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← TerrainHeightsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout depends on the lot's Version; the section has no version of its own. First come VertexGrid Count grids of (SizeX + 1) × (SizeZ + 1) 32-bit floats, one per tile corner, stored as SizeX + 1 rows of SizeZ + 1 values. Then, when HasWaterLevels, four 32-bit integers per tile, one per FloorQuarter, the tiles stored as SizeX rows of SizeZ.

When the lot's Version is less than 10 there is one vertex grid per level, indexed by level − LowestLevel, and the game skips all but the ground level's grid, which holds the terrain height. From version 10 there are two: grid 0 is the terrain height (the game's terrain height getter and setter use it, and in-game tests show that sculpting the terrain changes only this grid) and grid 1 is a second per-corner grid whose role is unknown; in-game tests with the raise, flatten and smooth tools changed only grid 0 and left grid 1 unchanged. The water levels are present from version 12; before it the game fills them with −400, which means no water.

Constructors

TerrainHeightsSection(int, int)

Initializes a new instance of the TerrainHeightsSection class in the layout of a lot whose Version is 12 or more, with two vertex grids and water levels, every vertex value 0 and every water level −400 (no water)

```
public TerrainHeightsSection(int sizeX, int sizeZ)
```

Parameters

`sizeX` int

The number of tiles along the lot's local X axis

`sizeZ` int

The number of tiles along the lot's local Z axis

Exceptions

ArgumentOutOfRangeException

`sizeX` or `sizeZ` is not 0 through 255

TerrainHeightsSection(int, int, int, bool)

Initializes a new instance of the TerrainHeightsSection class with every vertex value 0 and every water level −400 (no water)

```
public TerrainHeightsSection(int sizeX, int sizeZ, int vertexGridCount, bool hasWaterLevels)
```

Parameters

`sizeX` int

The number of tiles along the lot's local X axis

`sizeZ` int

The number of tiles along the lot's local Z axis

`vertexGridCount` int

The number of vertex grids: the lot's level count if the lot's Version is less than 10, otherwise 2

`hasWaterLevels` bool

Whether there are water levels, which there are if the lot's Version is 12 or more

Exceptions

ArgumentOutOfRangeException

`sizeX`, `sizeZ` or `vertexGridCount` is not 0 through 255

Properties

HasWaterLevels

Gets whether the section holds a water level per tile quarter, which it does when the lot's Version is 12 or more

```
public bool HasWaterLevels { get; }
```

Property Value

bool

SizeX

Gets the number of tiles along the lot's local X axis

```
public int SizeX { get; }
```

Property Value

int

SizeZ

Gets the number of tiles along the lot's local Z axis

```
public int SizeZ { get; }
```

Property Value

int

VertexGridCount

Gets the number of vertex grids

```
public int VertexGridCount { get; }
```

Property Value

int

Methods

GetVertexValue(int, int, int)

Gets the value of a tile corner in a vertex grid

```
public float GetVertexValue(int gridIndex, int x, int z)
```

Parameters

`gridIndex` int

The vertex grid: 0 for the terrain height when the lot's Version is 10 or more, otherwise the level − LowestLevel

`x` int

The corner's index along the lot's local X axis, 0 through SizeX

`z` int

The corner's index along the lot's local Z axis, 0 through SizeZ

Returns

float

Exceptions

ArgumentOutOfRangeException

A coordinate is outside the grid

GetWaterLevel(int, int, FloorQuarter)

Gets the water level of a quarter of a tile, in steps of 0.25 m

```
public int GetWaterLevel(int x, int z, FloorQuarter quarter)
```

Parameters

`x` int

The tile's index along the lot's local X axis

`z` int

The tile's index along the lot's local Z axis

`quarter` FloorQuarter

The quarter of the tile

Returns

int

Remarks

−400 (the game's default) means no water, and the value is −400 almost everywhere. The game multiplies it by 0.25 and uses the result minus the floor or terrain height as a water depth. In the lots examined it differs from −400 on every quarter of every pond and then equals the pond's WaterLevel; raising a pond's water level one step in game changed it from −3 to −2 on every quarter of the pond together with WaterLevel (in-game test).

Exceptions

InvalidOperationException

HasWaterLevels is false

ArgumentOutOfRangeException

A coordinate is outside the grid, or `quarter` is not a FloorQuarter

SetVertexValue(int, int, int, float)

Sets the value of a tile corner in a vertex grid

```
public void SetVertexValue(int gridIndex, int x, int z, float value)
```

Parameters

`gridIndex` int

The vertex grid (see GetVertexValue(int, int, int))

`x` int

The corner's index along the lot's local X axis, 0 through SizeX

`z` int

The corner's index along the lot's local Z axis, 0 through SizeZ

`value` float

The value

Exceptions

ArgumentOutOfRangeException

A coordinate is outside the grid

SetWaterLevel(int, int, FloorQuarter, int)

Sets the water level of a quarter of a tile, in steps of 0.25 m

```
public void SetWaterLevel(int x, int z, FloorQuarter quarter, int value)
```

Parameters

`x` int

The tile's index along the lot's local X axis

`z` int

The tile's index along the lot's local Z axis

`quarter` FloorQuarter

The quarter of the tile

`value` int

The water level, or −400 for no water (see GetWaterLevel(int, int, FloorQuarter))

Exceptions

InvalidOperationException

HasWaterLevels is false

ArgumentOutOfRangeException

A coordinate is outside the grid, or `quarter` is not a FloorQuarter

Class TerrainPaintLayer

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a layer of the TerrainPaintSection: a terrain paint and the mask of where it is applied

```
public sealed class TerrainPaintLayer
```

Inheritance

object ← TerrainPaintLayer

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The mask is stored as SizeX rows of SizeZ bytes.

Constructors

TerrainPaintLayer(int, int)

Initializes a new instance of the TerrainPaintLayer class with every mask value 0

```
public TerrainPaintLayer(int sizeX, int sizeZ)
```

Parameters

`sizeX` int

The number of mask values along the lot's local X axis, which must be the section's MaskSizeX

`sizeZ` int

The number of mask values along the lot's local Z axis, which must be the section's MaskSizeZ

Exceptions

ArgumentOutOfRangeException

`sizeX` or `sizeZ` is not 0 through 1,021, the mask size of a lot of 255 tiles

Properties

HasMask

Gets/sets whether the layer has a mask, written from version 3

```
public bool HasMask { get; set; }
```

Property Value

bool

Remarks

The game's writer writes true and the mask, or false and no mask, but its reader reads a mask either way, and so does this library. It is true in every lot examined.

this[int, int]

Gets/sets the mask value at a point

```
public byte this[int x, int z] { get; set; }
```

Parameters

`x` int

The index along the lot's local X axis

`z` int

The index along the lot's local Z axis

Property Value

byte

Exceptions

ArgumentOutOfRangeException

A coordinate is outside the mask

Key

Gets/sets the key of the paint, a CTProductTerrainPaint (0xEBCBB16C)

```
public ResourceKey Key { get; set; }
```

Property Value

ResourceKey

SizeX

Gets the number of mask values along the lot's local X axis

```
public int SizeX { get; }
```

Property Value

int

SizeZ

Gets the number of mask values along the lot's local Z axis

```
public int SizeZ { get; }
```

Property Value

int

UseCount

Gets/sets the number of uses of the layer

```
public uint UseCount { get; set; }
```

Property Value

uint

Remarks

The game's writer drops the layers whose count is 0. It equals the number of nonzero mask values in every layer examined; this library does not recompute it.

Class TerrainPaintSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the TerrainPaint section: the terrain paint layers, each a paint and a mask of where it is applied (🔓)

```
public sealed class TerrainPaintSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← TerrainPaintSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game's reader and writer keep two sets of layers, LayerSet0 and LayerSet1; what distinguishes them has not been established. The headers of the layers (key and use count) come first, set by set, then MaskSizeX and MaskSizeZ, then the masks, in the same order.

Versions 2 through 4 are supported, and the game writes version 4; the lots examined hold versions 2 and 4. Version 2 has only LayerSet1, followed by Unknown1, and its layers have no HasMask. Version 3 stores LayerSet1 before LayerSet0; version 4 stores LayerSet0 first.

The game writes no keys of this section into the ResourceManifestSection, which it saves earlier.

Constructors

TerrainPaintSection()

Initializes a new instance of the TerrainPaintSection class

```
public TerrainPaintSection()
```

Properties

LayerSet0

Gets the first set of layers, which version 2 does not have

```
public Collection<TerrainPaintLayer> LayerSet0 { get; }
```

Property Value

Collection<TerrainPaintLayer>

Remarks

When its caller passes a flag of unknown meaning, the game's reader replaces the key of this set's first layer with default key 0x1E of the lot's style. It holds 0 or 1 layers in the lots examined. Sculpting the terrain with the raise tool in game added a layer to this set (in-game test).

LayerSet1

Gets the second set of layers, the only one in version 2

```
public Collection<TerrainPaintLayer> LayerSet1 { get; }
```

Property Value

Collection<TerrainPaintLayer>

MaskSizeX

Gets/sets the number of mask values along the lot's local X axis

```
public int MaskSizeX { get; set; }
```

Property Value

int

Remarks

4 × SizeX + 1 in every lot examined, which this library requires, on decoding and on encoding, when it knows the lot's dimensions; the game itself reads only what fits its own mask.

MaskSizeZ

Gets/sets the number of mask values along the lot's local Z axis

```
public int MaskSizeZ { get; set; }
```

Property Value

int

Remarks

4 × SizeZ + 1 in every lot examined, which this library requires, on decoding and on encoding, when it knows the lot's dimensions.

Unknown1

Gets/sets a value of unknown meaning, written in version 2 only

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Remarks

The game reads and discards it; it equals the number of layers in every lot examined.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 2 through 4

Class Unknown7B33C1F5Section

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Unknown7B33C1F5 section: two lists of 64-bit values whose meaning has not been established (🔓)

```
public sealed class Unknown7B33C1F5Section : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← Unknown7B33C1F5Section

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Version 2 is the only one. Each list is preceded by a 16-bit count.

Only three of the lots examined hold this section: one lot of one world, in both its installed and its Maxis original form, and one lot of another world. The three are identical: FirstValues holds 20 values and SecondValues holds 1. Hypothesis: the values are object ids, because they resemble them, but none of them appears elsewhere in the same bundle.

Constructors

Unknown7B33C1F5Section()

Initializes a new instance of the Unknown7B33C1F5Section class

```
public Unknown7B33C1F5Section()
```

Properties

FirstValues

Gets the values of the first list

```
public Collection<ulong> FirstValues { get; }
```

Property Value

Collection<ulong>

SecondValues

Gets the values of the second list

```
public Collection<ulong> SecondValues { get; }
```

Property Value

Collection<ulong>

Class UnknownC9106475Entry

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one entry of an UnknownC9106475Section

```
public sealed class UnknownC9106475Entry
```

Inheritance

object ← UnknownC9106475Entry

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Id

Gets/sets the id of the entry, which the game uses as the key of a map

```
public ulong Id { get; set; }
```

Property Value

ulong

Levels

Gets the levels of the entry

```
public Collection<UnknownC9106475Level> Levels { get; }
```

Property Value

Collection <UnknownC9106475Level>

Class UnknownC9106475Level

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one level of an UnknownC9106475Entry

```
public sealed class UnknownC9106475Level
```

Inheritance

object ← UnknownC9106475Level

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Fields

KeyCount

Gets the number of keys of a level in version 3

```
public const int KeyCount = 6
```

Field Value

int

LegacyKeyCount

Gets the number of keys of a level before version 3

```
public const int LegacyKeyCount = 3
```

Field Value

int

Properties

Keys

Gets the keys, of which a version 3 section needs six and an older one three

```
public Collection<ResourceKey> Keys { get; }
```

Property Value

Collection <ResourceKey>

Remarks

The section stores each key instance first, then type, then group. Encoding fails when the number of keys is not the number the version stores.

Level

Gets/sets the lot level

```
public sbyte Level { get; set; }
```

Property Value

sbyte

Remarks

The section stores it as a 32-bit integer; the game keeps only a signed byte.

Class UnknownC9106475Section

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the UnknownC9106475 section: a list of entries, each holding levels with keys, whose meaning has not been established (🔓)

```
public sealed class UnknownC9106475Section : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← UnknownC9106475Section

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Versions 1 through 3 are supported, and the game writes version 3 (the game's reader accepts any version up to 3; the lower bound is a choice). Each level of an entry holds six keys from version 3, three before. The game discards the data after loading unless a flag is set.

Every lot examined holds version 3 with no entries, so the layout of the entries follows the game's reader and has not been observed.

Constructors

UnknownC9106475Section()

Initializes a new instance of the UnknownC9106475Section class

```
public UnknownC9106475Section()
```

Properties

Entries

Gets the entries

```
public Collection<UnknownC9106475Entry> Entries { get; }
```

Property Value

Collection<UnknownC9106475Entry>

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

> The version is not 1 through 3

Class VenueBusinessInfo

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one business entry of a VenueSection

```
public sealed class VenueBusinessInfo
```

Inheritance

object ← VenueBusinessInfo

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Key equals VenueTuning in all 50 entries examined, and differs from the section's own venue tuning in 13 of them.

Properties

Key

Gets/sets the key under which the game stores the entry

```
public ulong Key { get; set; }
```

Property Value

ulong

Payload

Gets/sets the message of the entry, serialized

```
public ReadOnlyMemory<byte> Payload { get; set; }
```

Property Value

ReadOnlyMemory <byte>

Remarks

The message type depends on the venue, so the bytes are kept as they are. A venue of 0x1FE99 uses the restaurant configuration message. In the entries examined the payloads of venue 0x53533 are empty, those of 0x1FE99 and 0x26C7F are not, and those of 0x6F417 are either.

VenueTuning

Gets/sets the tuning id of the venue the entry belongs to

```
public ulong VenueTuning { get; set; }
```

Property Value

ulong

Class VenueSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Venue section: the lot's venue (🔓)

```
public sealed class VenueSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← VenueSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Versions 3 through 6 are supported, and the game writes version 6. BusinessInfos exists from version 4, Unknown3 from version 5 and HouseholdId from version 6; Unknown1 and Unknown2 exist from versions 2 and 3, so in every supported version.

In tray and blueprint mode the game keeps only the entry of the current venue, so a record holds none or one entry.

Constructors

VenueSection()

Initializes a new instance of the VenueSection class

```
public VenueSection()
```

Properties

BusinessInfos

Gets the business entries, which exist from version 4

```
public Collection<VenueBusinessInfo> BusinessInfos { get; }
```

Property Value

Collection<VenueBusinessInfo>

HouseholdId

Gets/sets the id of the household, which exists from version 6

```
public ulong HouseholdId { get; set; }
```

Property Value

ulong

Remarks

The game turns a stored value of 1 into 0 on load. It is 0 in all but 5 of the 1,524 version 6 sections examined.

Unknown1

Gets/sets a flag

```
public bool Unknown1 { get; set; }
```

Property Value

bool

Remarks

Hypothesis: it marks a venue override, going by how the game's code uses it. It is 0 or 1 in the lots examined, and the game stores it in the lot's venue object.

Unknown2

Gets/sets a second flag, which the game stores next to Unknown1

```
public bool Unknown2 { get; set; }
```

Property Value

bool

Remarks

It is 0 in every lot examined.

Unknown3

Gets/sets a 32-bit value, which exists from version 5

```
public uint Unknown3 { get; set; }
```

Property Value

uint

Remarks

The game recomputes it when saving, from the WallBlocks section, and its meaning has not been established. In the 1,856 version 5 and 6 sections examined it has 331 distinct values and is 0 in 239.

VenueTuning

Gets/sets the tuning id of the lot's venue

```
public ulong VenueTuning { get; set; }
```

Property Value

ulong

Remarks

The game treats 0 as its default venue, 0x6FC6. Changing the lot type in game from Residential to Generic changed it from 0x6FC6 to 0x18201, and changing it back restored 0x6FC6 (in-game test). It is never 0 in the lots examined.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 3 through 6

Class VertexGraph

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents one of the graphs of vertices and edges of the VertexGraphsSection

```
public sealed class VertexGraph
```

Inheritance

object ← VertexGraph

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's writer; no code in the game's simulation library reads a graph back (other parts of the game were not examined). What each graph of the section describes is not established. Layout: Unknown1, the tag "TREV", an int count, the Vertices, the tag "TCES", an int count, then the Edges.

Properties

Edges

Gets the edges

```
public Collection<VertexGraphEdge> Edges { get; }
```

Property Value

Collection<VertexGraphEdge>

Unknown1

Gets/sets the value the game writes before the vertices

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Remarks

The game always writes 5; hypothesis: a version of the graph. 5 in every lot examined.

Vertices

Gets the vertices

```
public Collection<VertexGraphVertex> Vertices { get; }
```

Property Value

Collection<VertexGraphVertex>

Class VertexGraphEdge

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an edge of a VertexGraph

```
public sealed class VertexGraphEdge
```

Inheritance

object ← VertexGraphEdge

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Id, Vertex1Id, Vertex2Id, as the game's writer writes them.

Properties

Id

Gets/sets the id of the edge

```
public int Id { get; set; }
```

Property Value

int

Remarks

Its bytes appear to pack a level and kind, a direction and two coordinates (hypothesis); the same packing appears in the WallEdgesSection map keys.

Vertex1Id

Gets/sets the Id of the edge's first vertex, or -1 for none

```
public int Vertex1Id { get; set; }
```

Property Value

int

Vertex2Id

Gets/sets the Id of the edge's second vertex, or -1 for none

```
public int Vertex2Id { get; set; }
```

Property Value

int

Class VertexGraphVertex

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a vertex of a VertexGraph

```
public sealed class VertexGraphVertex
```

Inheritance

object ← VertexGraphVertex

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Id, Unknown1, Unknown2, Unknown3, as the game's writer writes them.

Properties

Id

Gets/sets the id of the vertex, by which Vertex1Id and Vertex2Id refer to it

```
public int Id { get; set; }
```

Property Value

int

Remarks

In every lot examined it is Unknown1 << 16 | Unknown2 << 8 | the byte of Unknown3, so it is derived from them; it is stored as read.

Unknown1

Gets/sets the first byte of the vertex

```
public byte Unknown1 { get; set; }
```

Property Value

byte

Remarks

Hypothesis: a coordinate along the lot's X axis. 1 through 63 in the lots examined.

Unknown2

Gets/sets the second byte of the vertex

```
public byte Unknown2 { get; set; }
```

Property Value

byte

Remarks

Hypothesis: a coordinate along the lot's Z axis. 1 through 63 in the lots examined.

Unknown3

Gets/sets the third byte of the vertex

```
public sbyte Unknown3 { get; set; }
```

Property Value

sbyte

Remarks

Hypothesis: a lot level. -4 through 4 in the lots examined.

Class VertexGraphsSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the VertexGraphs section: graphs of vertices and edges, whose version decides the layout of the WallEdgesSection (🔓)

```
public sealed class VertexGraphsSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← VertexGraphsSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Each graph holds vertices, under the tag "TREV" ("VERT" reversed), and edges, under the tag "TCES". What the graphs describe is not established; Sims 4 Studio calls this section WallCutout.

In in-game tests the first graph changed whenever walls, roofs, stairs or platforms were added or reshaped (including when a roof was replaced with another shape) and never for paint, trim, terrain or lot type changes; this is an observation only.

The layout follows the game's writer; on loading, the game reads only the section's version, which decides the layout of the WallEdgesSection, and does not read the graphs. Versions 5 through 9 are supported, and the game writes version 9; the lots examined hold versions 5 through 9.

Layout: Version, then the Graphs: three before version 8 and four from version 8 on, the numbers of graph objects the game's reader resets for those versions.

Constructors

VertexGraphsSection()

Initializes a new instance of the VertexGraphsSection class

```
public VertexGraphsSection()
```

Properties

Graphs

Gets the graphs

```
public Collection<VertexGraph> Graphs { get; }
```

Property Value

Collection<VertexGraph>

Remarks

Their number must match Version: three before version 8, four from version 8 on.

Version

Gets/sets the version of the section, which decides its layout and that of the WallEdgesSection

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when the number of Graphs does not match the version, or when a table or list of the WallEdgesSection that the layout of this version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 5 through 9

Class WallBlock

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a block: a room, platform, wall line, fence line, pool or fountain, as loops of segments on one level, with the catalog products along them

```
public sealed class WallBlock
```

Inheritance

object ← WallBlock

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

This is the game's WallInfo, which WallBlocksSection and PlexesSection both store; the layout follows the game's reader and writer. Versions 17 through 36 are supported, and the game writes version 36. The game reads versions 18 through 36; in the lots examined, version 17 appears only where the LotSize Section is version 2, and its layout is the game's layout for version 18 plus Unknown1. The lots examined hold versions 17, 22, 29, 31, 32, 33, 35 and 36; the layouts of the other versions follow the game's reader.

Layout, with version gates: Version (ushort), Id, Level, Unknown1 (17), BlockType (after 24), PlexId (after 35), HasFloor, Unknown3 (after 27), BaseElevation, FloorHeightOffset (after 33), Unknown4 (31 and 32), then before 27: Unknown5, Unknown6 (before 22), LegacyKeys (type, group, instance), Unknown7, Unknown8; then a ushort loop count (after 23; one loop before), the Loops, Unknown9 (before 19), Products (after 19), LegacyStamp (12 through 25), and before 27 Unknown10, Unknown11 and Unknown12.

Each segment stores these product slots: none before version 20 (the game takes all from LegacyKeys), slots 0 through 3 in versions 20 through 26 (the game takes slots 4 through 8 from LegacyKeys), 0 through 6 in versions 27 and 28, 0 through 7 in versions 29 and 32 through 34, and all nine in versions 30, 31, 35 and 36.

Properties

BaseElevation

Gets/sets the elevation of the block's base, in meters

```
public float BaseElevation { get; set; }
```

Property Value

float

Remarks

The game sets it to the elevation of Level plus the lot's offset; adding a foundation raised it by 0.25 (in-game test). It is −0.375 or −0.125 for fountains in the lots examined, and it was −3 for a new pool (in-game test).

BlockType

Gets/sets the type of the block

```
public WallBlockType BlockType { get; set; }
```

Property Value

WallBlockType

Remarks

Stored from version 25 on; the game treats older blocks as Standard.

FloorHeightOffset

Gets/sets the offset of the floor from BaseElevation, in meters

```
public float FloorHeightOffset { get; set; }
```

Property Value

float

Remarks

The game checks it with FloorHeightOffsetOutOfBounds before setting it and takes the block's bottom as the lower of BaseElevation and BaseElevation plus this offset. Raising a room's floor one step changed it from 0 to 0.25, and a platform had 0.25 when drawn and 0.5 after one raise (in-game tests); the game stores a platform as a block like a room, so a raised room floor and a platform differ in the data only by their segments, which are Open where a platform has no walls. Stored from version 34 on.

HasFloor

Gets/sets whether the block has a floor

```
public bool HasFloor { get; set; }
```

Property Value

bool

Remarks

Deleting the entire floor of a room changed only this value of the room's block, from true to false (in-game test). The game keeps it at offset 0x80 of the block, sets it to whether the level is at most 0 when it copies blocks into a plex and needs it for FloorHeightOffset. In the lots examined it is 1 for every pool and fountain block, for 36,076 of the 51,425 Standard blocks and for 5,271 of the 11,629 Fence blocks.

Id

Gets/sets the id of the block

```
public ushort Id { get; set; }
```

Property Value

ushort

Remarks

The game keys its blocks by this id and drops a block whose id is 0. The game renumbers the blocks when it saves: the same rooms had different ids from one save to the next (in-game test).

LegacyKeys

Gets the catalog keys that versions before 27 store for the whole block

```
public Collection<ResourceKey> LegacyKeys { get; }
```

Property Value

Collection <ResourceKey>

Remarks

Versions before 21 store 15 keys, versions 21 through 26 store 9. Of 15, the game uses positions 1, 2 and 5 for slots 0, 1 and 2, positions 3 and 4 for slot 7 (the later one wins), position 8 for slot 3 and positions 12 through 14 for slots 4 through 6; of 9, it uses positions 6 through 8 for slots 4 through 6. It reads the other positions as four uint values and discards them; in the lots examined they hold floor, window set and object catalog keys or the null key.

LegacyStamp

Gets/sets the value versions 12 through 25 store once for the whole block in place of each segment's FenceStamp and CeilingRailStamp

```
public uint LegacyStamp { get; set; }
```

Property Value

uint

Remarks

The game copies it into both values of every segment.

Level

Gets/sets the lot level of the block

```
public sbyte Level { get; set; }
```

Property Value

sbyte

Remarks

The game restricts the level by BlockType and takes the block's level height from it.

Loops

Gets the loops of the block

```
public Collection<WallBlockLoop> Loops { get; }
```

Property Value

Collection<WallBlockLoop>

Remarks

Versions before 24 hold exactly one loop. 1 through 12 in the lots examined.

PlexId

Gets/sets the id of the plex (unit) the block belongs to

```
public ushort PlexId { get; set; }
```

Property Value

ushort

Remarks

The game's plex manager sets it; 300 is the shared plex. The values match the ids of the PlexesSection plexes. Stored from version 36 on.

Products

Gets the table of the catalog products the segments refer to

```
public ProductTable Products { get; }
```

Property Value

ProductTable

Remarks

Stored from version 20 on.

Unknown1

Gets/sets a byte version 17 stores after Level

```
public byte Unknown1 { get; set; }
```

Property Value

byte

Remarks

It equals Level in every lot examined. The game's reader has no such byte.

Unknown10

Gets/sets a ushort versions before 27 store at the end of the block

```
public ushort Unknown10 { get; set; }
```

Property Value

ushort

Remarks

The game reads it and discards it. 4 in every lot examined.

Unknown11

Gets/sets a byte versions before 27 store after Unknown10

```
public byte Unknown11 { get; set; }
```

Property Value

byte

Remarks

The game reads it and discards it. 1 in every lot examined.

Unknown12

Gets/sets a byte versions before 27 store after Unknown11

```
public byte Unknown12 { get; set; }
```

Property Value

byte

Remarks

The game reads it and discards it. 1 in every lot examined.

Unknown3

Gets/sets a Boolean the game keeps at offset 0x81 of the block

```
public bool Unknown3 { get; set; }
```

Property Value

bool

Remarks

Hypothesis: whether the block has an automatic ceiling. The game sets it to whether every outer segment has a wall or the level is below 0, and clears it when it copies blocks into a plex. In the lots examined it is 0 for every Fence block, 1 for every fountain block, 1 for 466 of the 482 pool blocks and for 22,161 of the 51,182 Standard blocks that store it. In in-game tests a new fence block and a platform had 0 and enclosed rooms had 1, which is consistent with the hypothesis. Stored from version 28 on.

Unknown4

Gets/sets a byte versions 31 and 32 store after BaseElevation

```
public byte Unknown4 { get; set; }
```

Property Value

byte

Remarks

The game reads it and discards it. 0 in every version 31 block examined.

Unknown5

Gets/sets a byte versions before 27 store before LegacyKeys

```
public byte Unknown5 { get; set; }
```

Property Value

byte

Remarks

The game reads it and discards it. 2 in every lot examined.

Unknown6

Gets/sets a byte versions before 22 store before LegacyKeys

```
public byte Unknown6 { get; set; }
```

Property Value

byte

Remarks

The game reads it and discards it. 0 or 2 in the lots examined.

Unknown7

Gets/sets the first of two Booleans versions before 27 store after LegacyKeys

```
public bool Unknown7 { get; set; }
```

Property Value

bool

Remarks

The game reads it as a Boolean and discards it.

Unknown8

Gets/sets the second of two Booleans versions before 27 store after LegacyKeys

```
public bool Unknown8 { get; set; }
```

Property Value

bool

Remarks

The game reads it as a Boolean and discards it.

Unknown9

Gets/sets a byte versions before 19 store after the Loops

```
public byte Unknown9 { get; set; }
```

Property Value

byte

Remarks

The game's reader would discard it. 0 through 3 in the lots examined.

Version

Gets/sets the version of the block, which decides its layout

```
public ushort Version { get; set; }
```

Property Value

ushort

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

The version is not 17 through 36

Class WallBlockLoop

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a closed or open polygon of a WallBlock, as its segments

```
public sealed class WallBlockLoop
```

Inheritance

object ← WallBlockLoop

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

That the first loop of a block is its outer polygon and the rest are holes is a hypothesis from the docstring of the game's get_plex_outline Python function. Layout: a ushort segment count, the start points of all Segments, then the rest of each segment.

Properties

Segments

Gets the segments, in order around the loop

```
public Collection<WallBlockSegment> Segments { get; }
```

Property Value

Collection<WallBlockSegment>

Class WallBlockSegment

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a segment of a WallBlockLoop: the run from one point of the loop to the next, with the catalog products along it

```
public sealed class WallBlockSegment
```

Inheritance

object ← WallBlockSegment

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The products are ids into the block's Products table (0 for none); every nonzero id in the lots examined is an id of that table. The game reads them as nine slots; which slots a block version stores is described with Version, and the game fills the slots an older version does not store from the block's LegacyKeys. The slot names follow the catalog types the slots refer to in every lot examined (names from the game's CTProduct table).

Layout: the loop stores X and Z of all its segments first, then, per segment, Kind, Unknown1 (versions before 23), the stored slots in slot order, then FenceStamp and CeilingRailStamp (versions after 25).

Properties

CeilingRailProductId

Gets/sets the id of the ceiling rail (CTProductCeilingRail 0x3F0C529A), slot 3

```
public uint CeilingRailProductId { get; set; }
```

Property Value

uint

Remarks

This is the product other tools call a spandrel.

CeilingRailStamp

Gets/sets the value the game keeps with the ceiling rail slot

```
public uint CeilingRailStamp { get; set; }
```

Property Value

uint

Remarks

The game returns it with slot 3. Adding a foundation set it, together with FenceStamp, to the value StampCounter was raised to (in-game test); what the game uses it for is unknown. It equals FenceStamp in 66% of the segments of the WallBlocksSection blocks examined and in 81% of those of the Plexes Section blocks. Stored from version 26 on; older versions store one value for the whole block, Legacy Stamp.

FenceProductId

Gets/sets the id of the fence (Fence 0x0418FE2A) or half wall (CTProductWall 0x9151E6BC), slot 2

```
public uint FenceProductId { get; set; }
```

Property Value

uint

Remarks

The game emits CantPlacePoolUnderFence when a pool would go under a segment with this slot set.

FenceStamp

Gets/sets the value the game keeps with the fence slot

```
public uint FenceStamp { get; set; }
```

Property Value

uint

Remarks

The game returns it with slot 2. Adding a foundation set it, together with CeilingRailStamp, to the value StampCounter was raised to (in-game test); what the game uses it for is unknown. Stored from version 26 on; older versions store one value for the whole block, LegacyStamp.

FloorTrimProductId

Gets/sets the id of the floor trim (0x84C23219), slot 6

```
public uint FloorTrimProductId { get; set; }
```

Property Value

uint

Remarks

Never 0 in the lots examined.

FoundationProductId

Gets/sets the id of the foundation (0x2FAE983E), slot 4

```
public uint FoundationProductId { get; set; }
```

Property Value

uint

Remarks

The game requires the same foundation on all segments of a block, otherwise InconsistentFoundationTypes. Never 0 in the lots examined.

FriezeProductId

Gets/sets the id of the frieze (0xA057811C), slot 5

```
public uint FriezeProductId { get; set; }
```

Property Value

uint

Remarks

Putting a frieze on one wall in game set this slot of that wall's segment to the chosen frieze and cleared the default frieze (0x1F1E) from the room's other segments; painting one tile of that wall afterwards left the chosen frieze in place (in-game tests).

Kind

Gets/sets the kind of the segment

```
public WallBlockSegmentKind Kind { get; set; }
```

Property Value

WallBlockSegmentKind

Remarks

Stored as read; see WallBlockSegmentKind for how the game maps the values of versions before 27.

PlatformTrimProductId

Gets/sets the id of the platform trim (CTProductPlatformTrim 0x99C07284), slot 8

```
public uint PlatformTrimProductId { get; set; }
```

Property Value

uint

Remarks

Changing a platform's edge trim in game changed only this slot (in-game test). When it reads a pool block of version 30 or 31, the game copies this slot into TrimProductId. 0 in every version 31 block examined.

SideAWallPatternId

Gets/sets the id of the wall pattern (CTProductWallPattern 0xD5F0F921) of one side, slot 0

```
public uint SideAWallPatternId { get; set; }
```

Property Value

uint

Remarks

Painting one wall tile split its segment out and gave this slot a new wall pattern in one in-game test, and SideBWallPatternId in another: which of the two slots changes depends on the face painted.

SideBWallPatternId

Gets/sets the id of the wall pattern (CTProductWallPattern 0xD5F0F921) of the other side, slot 1

```
public uint SideBWallPatternId { get; set; }
```

Property Value

uint

Remarks

Painting one wall tile split its segment out and gave this slot a new wall pattern in one in-game test, and SideAWallPatternId in another: which of the two slots changes depends on the face painted. That this is the side opposite SideAWallPatternId is a hypothesis from the slot's catalog type.

TrimProductId

Gets/sets the id of the trim, slot 7: a half wall trim (CTProductHalfWallTrim 0xFE33068E), pool trim (0xA5DFFCF3) or fountain trim (0xE7ADA79D)

```
public uint TrimProductId { get; set; }
```

Property Value

uint

Remarks

In the lots examined fountain trims appear only in Fountain blocks, half wall trims only in Standard and Pool blocks, and pool trims mostly in Pool blocks (2,267 of 2,785 segments) and otherwise in Standard and Fountain blocks.

Unknown1

Gets/sets a byte versions before 23 store after Kind

```
public byte Unknown1 { get; set; }
```

Property Value

byte

Remarks

The game reads it and discards it.

X

Gets/sets the coordinate of the segment's start point along the lot's X axis

```
public byte X { get; set; }
```

Property Value

byte

Z

Gets/sets the coordinate of the segment's start point along the lot's Z axis

```
public byte Z { get; set; }
```

Property Value

byte

Enum WallBlockSegmentKind

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Specifies the kind of a WallBlockSegment

```
[SuppressMessage("Design", "CA1028: Enum Storage should be Int32", Justification = "The game stores the value as a byte.")]
public enum WallBlockSegmentKind : byte
```

Fields

`ModularPiece = 2`

A segment of a modular piece without a wall

`ModularPieceWithWall = 3`

A segment of a modular piece with a wall

`Open = 1`

A segment without a wall

`Wall = 0`

A segment with a wall

Remarks

The game treats a block as enclosed when none of the segments of its outer loop is Open or ModularPiece, emits CantPlacePoolUnderWall for a Wall segment, and requires ModularPiece or ModularPieceWithWall on a diagonal segment flanked by 45° segments of the same kind, otherwise InvalidBlockModularPieceConfiguration. That such segments form rounded corners is a hypothesis. Before block version 27 the game maps the stored values 0 through 3 to 0, 1, 2 and 1 with a table.

Enum WallBlockType

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Specifies the type of a WallBlock

```
[SuppressMessage("Design", "CA1028: Enum Storage should be Int32", Justification = "The game
stores the value as a byte.")]
public enum WallBlockType : byte
```

Fields

`Fence = 1`

A fence line

Placing a fence creates a block of this type (in-game test). The game allows it only at level 0 and gives it no elevation.

`Fountain = 2`

A fountain

The game emits CantPlaceFountainUnderAnything for it, gives it a height of 0.375 and allows it only at level -1.

`Pool = 3`

A pool

The game emits CantPlacePoolUnderOtherPool for it; placing a pool creates a block of this type (in-game test).

`Standard = 0`

A block that is not a fence, fountain or pool

In the lots examined these are rooms and free-standing wall lines, and a platform is a block of this type too: drawing one creates a block whose segments are all Open until walls are added, raised by FloorHeightOffset just as a room whose floor is raised (in-game tests); that the type covers nothing else is a hypothesis.

Remarks

The game checks each block type against the WallBlockSegmentKind of the block's segments with a table: a Fence block allows only Open, a Fountain or Pool block only Wall and ModularPieceWithWall (otherwise BlockTypeIncompatibleWithProduct).

Class WallBlocksSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the WallBlocks section: the lot's blocks, which are its rooms, wall lines, fence lines, pools and fountains (🔓)

```
public sealed class WallBlocksSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← WallBlocksSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The layout follows the game's reader and writer. Versions 0 through 2 are supported, and the game writes version 2; the lots examined hold only version 2.

Layout: Version, StampCounter (after 1), a ushort block count, then per block its Id (ushort, the key of the game's block map, which the game writes from the block) and the WallBlock.

Constructors

WallBlocksSection()

Initializes a new instance of the WallBlocksSection class

```
public WallBlocksSection()
```

Properties

Blocks

Gets the blocks

```
public Collection<WallBlock> Blocks { get; }
```

Property Value

Collection<WallBlock>

StampCounter

Gets/sets the counter from which the game stamps FenceStamp and CeilingRailStamp

```
public uint StampCounter { get; set; }
```

Property Value

uint

Remarks

The game keeps it at offset 0x58 of its block manager. Adding a foundation raised it by 1 and set the stamps of the block's segments to the new value (in-game test). It is at least the highest Id in every lot examined; of the 653 lots examined without blocks, 450 hold 0 and the rest 1 through 143,271. Whether the game also allocates block ids from it is a hypothesis. Stored from version 2 on.

Version

Gets/sets the version of the section, which decides its layout

```
public uint Version { get; set; }
```

Property Value

uint

Remarks

Encoding throws an InvalidOperationException when a field the version lacks holds a value other than its default, or a collection the version lacks is not empty.

Exceptions

ArgumentOutOfRangeException

> The version is not 0 through 2

Class WallEdgePatterns

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of WallPatternEdges: the two wall patterns of a wall edge

```
public sealed class WallEdgePatterns
```

Inheritance

object ← WallEdgePatterns

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: MapKey, WallPattern1Id, WallPattern2Id, as the game's reader reads them.

Properties

MapKey

Gets/sets the key under which the game maps the entry

```
public uint MapKey { get; set; }
```

Property Value

uint

Remarks

Hypothesis: it identifies a wall edge, its bytes packing the level (high nibble of byte 0, signed) and a kind (low nibble), a direction (byte 1: 0, 1, 6 or 7) and two coordinates (bytes 2 and 3). About 42% of the keys in the lots examined are also Id values of the bundle's VertexGraphsSection.

WallPattern1Id

Gets/sets the Id of one wall pattern in WallPatterns, or 0 for none

```
public ushort WallPattern1Id { get; set; }
```

Property Value

ushort

Remarks

That the two ids are the patterns of the two sides of the wall is a hypothesis from their catalog type (CTProductWallPattern). In 99.79% of the entries examined both are ids of the table or 0.

WallPattern2Id

Gets/sets the Id of the other wall pattern in WallPatterns, or 0 for none

```
public ushort WallPattern2Id { get; set; }
```

Property Value

ushort

Class WallEdgeProduct

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of ProductEdges or AdditionalProductEdges: the catalog product along a wall edge

```
public sealed class WallEdgeProduct
```

Inheritance

object ← WallEdgeProduct

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: MapKey, ProductId, as the game's reader reads them.

Properties

MapKey

Gets/sets the key under which the game maps the entry

```
public uint MapKey { get; set; }
```

Property Value

uint

Remarks

It is packed like MapKey (hypothesis). In ProductEdges the low nibble of byte 0 follows the product's catalog type in the lots examined: 0 for CTProductWall, 1 for ceiling rails, 2 for fences and railings, and 3 for some fences.

ProductId

Gets/sets the Id of the product in the table that goes with the list

```
public uint ProductId { get; set; }
```

Property Value

uint

Remarks

In AdditionalProductEdges it is 0 in 99% of the entries examined.

Class WallEdgesLegacyEntry

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents an entry of LegacyEntries

```
public sealed class WallEdgesLegacyEntry
```

Inheritance

object ← WallEdgesLegacyEntry

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Layout: Unknown1, Unknown2. The game reads both and discards them.

Properties

Unknown1

Gets/sets the uint of the entry

```
public uint Unknown1 { get; set; }
```

Property Value

uint

Unknown2

Gets/sets the byte of the entry

```
public byte Unknown2 { get; set; }
```

Property Value

byte

Class WallEdgesSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the WallEdges section: the wall patterns and catalog products along the lot's wall edges (🔓)

```
public sealed class WallEdgesSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← WallEdgesSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game reads and writes the section right after the VertexGraphsSection, whose Version decides the layout; so the section can only be read and written together with that section. The section has no version of its own; the game writes it for a VertexGraphsSection of version 9.

Layout: a uint count, the WallPatternEdges and the WallPatterns table; if the vertex graphs version is above 1, the Products table, a uint count and the ProductEdges; if it is 6, a uint count and the Legacy Entries; if it is above 8, the AdditionalProducts table, a uint count and the AdditionalProductEdges.

Encoding throws an InvalidOperationException when a table or list that the layout of the vertex graphs version lacks is not empty.

Other tools detect the optional parts by the length that remains; in every lot examined that gives the same split as the version gates.

Constructors

WallEdgesSection()

Initializes a new instance of the WallEdgesSection class

```
public WallEdgesSection()
```

Properties

AdditionalProductEdges

Gets the entries of the third product map

```
public Collection<WallEdgeProduct> AdditionalProductEdges { get; }
```

Property Value

Collection<WallEdgeProduct>

Remarks

What the map is for is unknown. Stored when the vertex graphs version is above 8.

AdditionalProducts

Gets the table of the catalog products AdditionalProductEdges refers to

```
public ProductTable AdditionalProducts { get; }
```

Property Value

ProductTable

Remarks

It holds at most 3 keys, all CTProductWall (0x9151E6BC), in the lots examined. Stored when the vertex graphs version is above 8.

LegacyEntries

Gets the entries of a list the game reads and discards

```
public Collection<WallEdgesLegacyEntry> LegacyEntries { get; }
```

Property Value

Collection<WallEdgesLegacyEntry>

Remarks

Stored when the vertex graphs version is 6; two lots examined hold entries (1,284 in all).

ProductEdges

Gets the catalog products along the wall edges

```
public Collection<WallEdgeProduct> ProductEdges { get; }
```

Property Value

Collection<WallEdgeProduct>

Remarks

Stored when the vertex graphs version is above 1.

Products

Gets the table of the catalog products ProductEdges refers to

```
public ProductTable Products { get; }
```

Property Value

ProductTable

Remarks

In the lots examined it holds CTProductWall (0x9151E6BC), fence (0x0418FE2A), ceiling rail (CTProductCeilingRail 0x3F0C529A) and railing (0x1C1CF1F7) keys. Stored when the vertex graphs

version is above 1.

WallPatternEdges

Gets the wall patterns of the wall edges

```
public Collection<WallEdgePatterns> WallPatternEdges { get; }
```

Property Value

Collection <WallEdgePatterns>

WallPatterns

Gets the table of the wall patterns (CTProductWallPattern 0xD5F0F921) WallPatternEdges refers to

```
public ProductTable WallPatterns { get; }
```

Property Value

ProductTable

Class WeatherSection

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents the Weather section: the lot's seasonal weather interpolation value (🔓)

```
public sealed class WeatherSection : LotArchitectureSection
```

Inheritance

object ← LotArchitectureSection ← WeatherSection

Inherited Members

LotArchitectureSection.Type , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Value is the value of the game's `request_season_weather_interpolation` Python function for interpolation type 1; which weather quantity that is has not been established. It is 0 in 2,087 of the 2,102 sections examined and up to about 0.86 in the rest. Version 1 is the only one.

Constructors

WeatherSection()

Initializes a new instance of the WeatherSection class

```
public WeatherSection()
```

Properties

Value

Gets/sets the interpolation value

```
public float Value { get; set; }
```

Property Value

float

Class WorldLotObjectsModel

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Represents a WorldLotObjects resource: the objects of a lot template in a world file, as a `LotObjectList` protobuf message in protobuf text format (🔓)

```
public sealed class WorldLotObjectsModel : Model, IModel<WorldLotObjectsModel>, IModel
```

Inheritance

object ← Model ← WorldLotObjectsModel

Implements

IModel<WorldLotObjectsModel>, IModel

Inherited Members

Model.Decode(ResourceType, ReadOnlyMemory<byte>) , Model.GetModelType(ResourceType) , Model.GetName(ResourceType, Stream) , Model.GetNameAsync(ResourceType, Stream, CancellationToken) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game names the resource after the lot template it belongs to; see GetKey(uint). Its WorldLot Architecture counterpart holds the lot template's architecture.

Maxis writes the text with fields in field-number order, two-space indentation, CRLF line endings and floats in .NET Framework's round-trip format; Encode() writes the same, so it reproduces every resource Maxis wrote byte for byte. Decode(ReadOnlyMemory<byte>) also reads text written otherwise (community tools add commas, reorder fields or write hexadecimal integers), and Encode() returns the decoded bytes unchanged as long as LotObjects is unchanged.

Constructors

WorldLotObjectsModel()

Initializes a new instance of the WorldLotObjectsModel class with no objects

```
public WorldLotObjectsModel()
```

Properties

LotObjects

Gets/sets the lot template's objects

```
public LotObjectList LotObjects { get; set; }
```

Property Value

LotObjectList

ResourceName

Gets the name of this resource if it has one

```
public override string? ResourceName { get; }
```

Property Value

string

SupportedTypes

Gets a list of resource types that this model supports

```
public static ISet<ResourceType> SupportedTypes { get; }
```

Property Value

ISet<ResourceType>

Methods

Decode(ReadOnlyMemory<byte>)

Decodes the resource in raw format to produce an operable model (🔄 🏃)

```
public static WorldLotObjectsModel Decode(ReadOnlyMemory<byte> data)
```

Parameters

`data` ReadOnlyMemory<byte>

Returns

WorldLotObjectsModel

Exceptions

InvalidDataException

The data is not protobuf text format of a `LotObjectList` message

Encode()

Encodes the resource model into raw format (🔄 🏃)

```
public override ReadOnlyMemory<byte> Encode()
```

Returns

ReadOnlyMemory<byte>

GetKey(uint)

Gets the key of the WorldLotObjects resource of the lot template with the specified `lotTemplateId`

```
public static ResourceKey GetKey(uint lotTemplateId)
```

Parameters

`lotTemplateId` uint

Returns

ResourceKey

Remarks

The instance is that of the lot template's WorldLotArchitecture resource (see GetLotTemplateInstance(uint)), and the type is the FNV-1 32-bit hash of the extension `lottemp`.

GetName(Stream)

Gets the name of a resource from its raw data, if it has one (🔄 💤)

```
public static string? GetName(Stream stream)
```

Parameters

`stream` Stream

Returns

string

GetNameAsync(Stream, CancellationToken)

Gets the name of a resource from its raw data, if it has one, asynchronously (🔄 💤)

```
public static Task<string?> GetNameAsync(Stream stream, CancellationToken cancellationToken
= default)
```

Parameters

`stream` Stream

`cancellationToken` CancellationToken

Returns

Task<string>

Class ZoneArchitecture

Namespace: LlamaLogic.Architecture

Assembly: LlamaLogic.Architecture.dll

Provides access to the architecture of the lots of a zone in a save game: the `architecture_data` field of the `EA.Sims4.Persistence.ZoneObjectData` protobuf message, the content of a ZoneObjectData resource

```
public static class ZoneArchitecture
```

Inheritance

object ← ZoneArchitecture

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

`architecture_data` is a repeated field parallel to the message's `objects` field, and `active_data_index` selects the entry in use. Most zones have one entry, some have none, and a few have several.

These methods read and replace only that field's entries in the message's bytes and copy everything else verbatim, so they never re-serialize the rest of the message.

Methods

GetArchitecture(ReadOnlyMemory<byte>)

Gets the architecture of each entry of the `architecture_data` field of a `ZoneObjectData` message, or null for an empty entry

```
public static IReadOnlyList<LotArchitecture?> GetArchitecture(ReadOnlyMemory<byte>
zoneObjectData)
```

Parameters

`zoneObjectData` ReadOnlyMemory<byte>

Returns

IReadOnlyList<LotArchitecture>

Exceptions

InvalidDataException

The data is not a well-formed protobuf message, or an entry is not a well-formed lot architecture

GetArchitectureData(ReadOnlyMemory<byte>)

Gets the content of each entry of the `architecture_data` field of a `ZoneObjectData` message: a record bundle, or nothing

```
public static IReadOnlyList<ReadOnlyMemory<byte>> GetArchitectureData(ReadOnlyMemory<byte>
zoneObjectData)
```

Parameters

`zoneObjectData` ReadOnlyMemory<byte>

Returns

IReadOnlyList<ReadOnlyMemory<byte>>

Exceptions

InvalidDataException

The data is not a well-formed protobuf message

SetArchitecture(ReadOnlyMemory<byte>, IReadOnlyList<LotArchitecture?>)

Gets a `ZoneObjectData` message with each entry of its `architecture_data` field replaced by the encoding of the corresponding `architecture`, or emptied where it is null

```
public static ReadOnlyMemory<byte> SetArchitecture(ReadOnlyMemory<byte> zoneObjectData,
IReadOnlyList<LotArchitecture?> architecture)
```

Parameters

`zoneObjectData` ReadOnlyMemory<byte>

`architecture` IReadOnlyList<LotArchitecture>

Returns

ReadOnlyMemory<byte>

Exceptions

InvalidDataException

The data is not a well-formed protobuf message

ArgumentException

The number of entries of `architecture` differs from that of the message's field

InvalidOperationException

An architecture cannot be encoded as it is

SetArchitectureData(ReadOnlyMemory<byte>, IReadOnlyList<ReadOnlyMemory<byte>>)

Gets a `ZoneObjectData` message with the content of each entry of its `architecture_data` field replaced by the corresponding `architectureData`

```
public static ReadOnlyMemory<byte> SetArchitectureData(ReadOnlyMemory<byte> zoneObjectData,
IReadOnlyList<ReadOnlyMemory<byte>> architectureData)
```

Parameters

`zoneObjectData` ReadOnlyMemory<byte>

`architectureData` IReadOnlyList<ReadOnlyMemory<byte>>

Returns

ReadOnlyMemory<byte>

Exceptions

InvalidDataException

The data is not a well-formed protobuf message

ArgumentException

The number of entries of `architectureData` differs from that of the message's field